

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA HÀ NỘI



BÁO CÁO MÔN HỌC

Dự đoán giá điện thoại bằng mô hình học máy

Môn học: Học máy

Giảng viên hướng dẫn: TS. Cao Văn Chung

Nhóm thực hiện:

Nguyễn Cảnh Tùng - MSSV: 23001950

Nguyễn Ngọc Tuấn - MSSV: 23001948

Nguyễn Khánh Toàn - MSSV: 23001944

Hà Nội, tháng 1 năm 2026

Contents

I	Giới thiệu bài toán và phương pháp	2
1	Giới thiệu bài toán	2
2	Phương pháp	3
II	Tiền xử lý dữ liệu	3
III	Cơ sở lý thuyết	7
1	Principal component analysis	7
2	Linear discriminant analysis	8
3	Các phương pháp phân cụm	10
a)	Phương pháp K trung bình	10
b)	Phương pháp DBScan	10
c)	Mô hình trộn Gaussian - GMM	11
4	K-NN Method	12
5	Hồi quy tuyến tính	13
6	Các bài toán phân loại	14
a)	Phương pháp K-NN cho bài toán phân loại nhiều lớp	14
b)	Gaussian Naive Bayes (GNB)	15
c)	Bernoulli Naive Bayes (BNB)	15
d)	Phương pháp Hồi quy SoftMax	16
7	Validation	17
a)	Validation	17
b)	Cross-validation	19
IV	Phân tích và trực quan hóa dữ liệu	19
1	Phân tích và trực quan với PCA	21
2	Phân tích và trực quan với LDA	24
V	Phân cụm dữ liệu	27
1	Phương pháp DBScan	27
a)	Kết quả	27
b)	Đánh giá	28
c)	Trực quan hóa	29
2	Phương pháp GMM	30
a)	Lựa chọn số cụm tối ưu (k)	30
b)	Kết quả và Đánh giá	31
c)	Trực quan hóa dữ liệu	31

3	Phương pháp Kmeans	32
a)	Thiết lập thực nghiệm	32
b)	Lựa chọn số cụm tối ưu (k)	32
c)	Kết quả và Phân tích	33
d)	Trực quan hóa trên không gian giảm chiều	34
VI Kết quả thực nghiệm		34
1	Các bài toán hồi quy	34
a)	Mô hình K láng giềng gần nhất (K-NN)	35
b)	Mô hình Hồi quy tuyến tính	36
c)	Nhận xét, so sánh giữa 2 mô hình	40
2	Các bài toán phân loại	41
a)	Mô hình Naive Bayes	42
b)	Mô hình hồi quy SoftMax	43
VII Tổng kết và Hướng phát triển		44
1	Tóm tắt kết quả	44
2	Phân tích và Đánh giá	45
a)	Hiệu suất Linear Regression	45
b)	Hiệu suất KNN	45
c)	Hiệu suất Naive Bayes	45
d)	Hiệu suất SoftMax	45
3	Kết luận	46
4	Hướng phát triển	46

I Giới thiệu bài toán và phương pháp

1 Giới thiệu bài toán

Trong thị trường điện thoại thông minh hiện nay, giá sản phẩm phụ thuộc vào nhiều yếu tố kỹ thuật như dung lượng bộ nhớ ngoài, bộ nhớ trong, kích thước màn hình, độ phân giải camera hay dung lượng pin. Việc dự đoán giá điện thoại dựa trên các thông số kỹ thuật không chỉ giúp người tiêu dùng so sánh và lựa chọn sản phẩm phù hợp mà còn hỗ trợ các nhà bán lẻ trong việc định giá hợp lý.

Bài toán đặt ra là xây dựng một mô hình học máy có khả năng **dự đoán giá điện thoại** dựa trên các đặc trưng đầu vào kể trên. Mục tiêu là tìm ra mối quan hệ giữa thông số kỹ thuật và giá bán, đồng thời đạt được độ chính xác cao nhất có thể trên tập dữ liệu thử nghiệm.

2 Phương pháp

Để giải quyết bài toán dự đoán giá điện thoại, nhóm áp dụng quy trình gồm các bước: thu thập dữ liệu, tiền xử lý dữ liệu, huấn luyện mô hình và đánh giá kết quả.

Do chưa có điều kiện để trực tiếp thu thập dữ liệu từ thực tế nên nhóm sử dụng bộ dữ liệu có sẵn được tải về và sử dụng từ trang Kaggle. Bộ dữ liệu này chứa dữ liệu được thu thập và xử lý sơ bộ từ trang web Gadgets360 bởi người đăng tải bộ dữ liệu. Chi tiết về cách thu thập và xử lý sơ bộ của người đăng tải có thể xem tại trang web của bộ dữ liệu có địa chỉ Kaggle Mobile Price Dataset.

Dữ liệu được nhóm tiền xử lý loại bỏ các trường không phù hợp, kiểm tra các giá trị lỗi, thiếu. Thực hiện chuyển đổi định dạng một số trường phân loại với one hot encoding. Rồi tiến hành đưa vào mô hình thực hiện các bước tiếp theo.

Nhóm sử dụng hai thuật toán học máy cơ bản là **K-Nearest Neighbors (KNN)** và **Linear Regression**. Thuật toán KNN được lựa chọn nhờ tính đơn giản, xác định giá trị dự đoán của một điểm dữ liệu mới dựa trên trung bình của K điểm lân cận gần nhất. Vì KNN không thực sự “học” từ dữ liệu mà thực hiện tính toán tại thời điểm dự đoán, nhóm sử dụng kỹ thuật *cross-validation* để đánh giá mô hình khách quan hơn và lựa chọn giá trị K tối ưu, nhằm cân bằng giữa hiện tượng *overfitting* và *underfitting*.

Nhóm sử dụng mô hình **Linear Regression** để *hồi quy giá điện thoại* dựa trên các thuộc tính kỹ thuật đầu vào như dung lượng pin, kích thước màn hình, bộ nhớ, vi xử lý và hệ điều hành. Mô hình được lựa chọn nhờ tính đơn giản, dễ diễn giải và khả năng phản ánh mối quan hệ tuyến tính giữa các biến độc lập và giá bán.

Nhóm áp dụng phương pháp **Principal Component Analysis (PCA)** để *giảm chiều dữ liệu* trước khi huấn luyện mô hình, nhằm loại bỏ nhiễu và các thuộc tính dư thừa. Đồng thời, mô hình cũng được huấn luyện trên *dữ liệu gốc* để so sánh hiệu quả giữa hai cách tiếp cận.

II Tiền xử lý dữ liệu

Bộ dữ liệu gồm 1359 mẫu, 21 trường (không tính trường id) gồm 2 trường tên và mẫu, 9 trường kiểu phân loại và 9 trường kiểu số, cuối cùng là trường giá bán. Cụ thể các trường được trình bày ở bảng dưới.

Table 1: Mô tả các thuộc tính trong bộ dữ liệu

Tên thuộc tính	Mô tả	Kiểu dữ liệu
Name / Model	Tên và loại của máy. Không cần thiết cho mô hình dự đoán.	Chữ
Brand	Tên công ty sản xuất điện thoại.	Phân loại (Categorical)
Battery capacity (mAh)	Dung lượng pin của điện thoại.	Số (Numeric)
Screen size (inches), Resolution x, Resolution y	Kích thước và độ phân giải màn hình.	Số (Numeric)
Touchscreen	Màn hình cảm ứng (Yes nếu có, No nếu không).	Phân loại (Categorical)
Processor	Số nhân xử lý của vi xử lý (CPU cores).	Số (Numeric)
RAM (MB)	Dung lượng bộ nhớ RAM.	Số (Numeric)
Internal storage (GB)	Dung lượng bộ nhớ lưu trữ trong.	Số (Numeric)
Rear camera, Front camera	Độ phân giải camera sau và trước.	Số (Numeric)
Operating system	Hệ điều hành của thiết bị (Android, iOS, v.v.).	Phân loại (Categorical)
Wi-Fi, Bluetooth, GPS, Number of SIMs, 3G, 4G/LTE	Tính năng kết nối và hỗ trợ (Yes là có, No là không).	Phân loại (Categorical)
Price	Giá bán của điện thoại (Rupee Ấn Độ)	Số (Numeric)

Bộ dữ liệu được nhóm tiền xử lý sơ bộ bao gồm loại bỏ trường không cần thiết cho mô hình bài toán là trường Name và Model, đây là các trường chứa tên của điện thoại, không có giá trị trong bài toán dự đoán giá dựa vào thông số kỹ thuật. Nhóm đổi đơn vị tính của trường RAM (MB) thành GB bằng cách chia cho 1000. Ở đây chia cho 1000 thay vì 1024 bởi, vốn trong kỹ thuật máy tính thì $1\text{GB} = 1024\text{MB}$ nhưng trong quảng cáo và cách tính của các công ty điện thoại thì tính $1\text{GB} = 1000\text{MB}$. Trường Price đang ở đơn vị Rupee của Ấn Độ (INR) được đổi về Đô la (USD) bằng cách nhân với 0.011, tại thời điểm của bài báo cáo này được viết thì 1 INR bằng 0.011 USD. Dữ liệu được kiểm tra các giá trị thiếu, không hợp lệ, nếu có thì tiến hành loại bỏ.

Nhóm tiến hành vẽ đồ thị của `Price` để xem phân bố của nó. Phân bố bị lệch phải, cho thấy dữ liệu rất mất cân bằng. Một số ít mẫu có giá cao trong khi đó phần lớn dữ liệu thì đều có giá quanh khoảng 220 làm cho đồ thị kéo dài về phía phải. Điều này sẽ ảnh hưởng xấu đến các mô hình nhạy cảm với phân phối như Linear Regression.

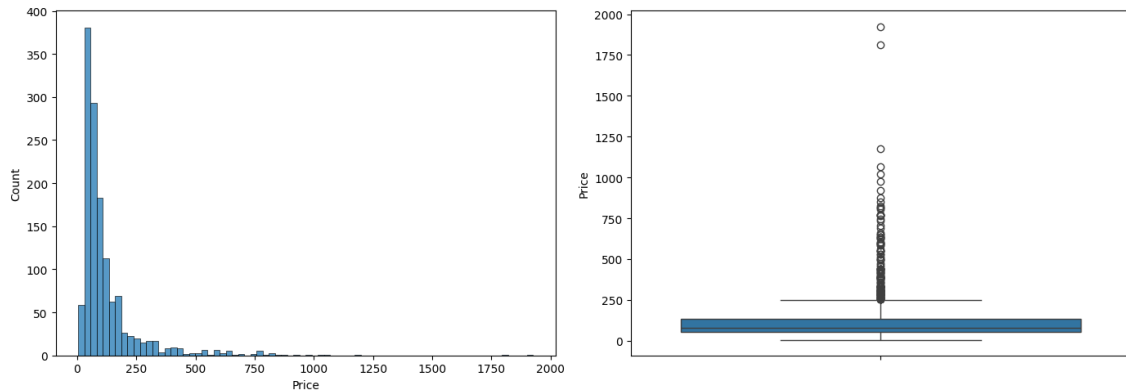


Figure 1: Biểu đồ phân bố giá (`Price`) trong bộ dữ liệu.

Tiến hành dùng Log-transform đối với `Price`. Đưa dữ liệu của mục tiêu gần với phân phối chuẩn hơn. Giúp cho mô hình Linear Regression hoạt động tốt hơn. Thêm vào đó loại bỏ các giá trị ngoại lai (có $zscore \geq 3$), những giá trị lớn hơn giá trị trung bình 3 lần độ lệch chuẩn.

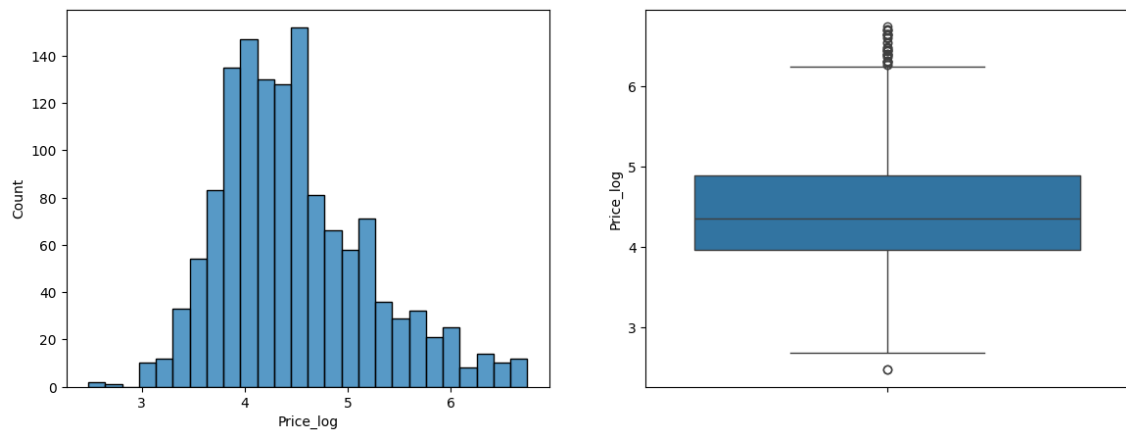


Figure 2: Biểu đồ phân bố giá sau khi Log Transform.

Tiến hành vẽ biểu đồ của các trường phân loại `Touchscreen`, `Wi-Fi`, `Bluetooth`, `GPS`, `3G`, và `4G`. Đồ thị của các trường này rất lệch. Mô hình khó có thể học được sự khác biệt về giá dựa vào những trường này. Nhóm loại bỏ các trường phân loại này, chỉ giữ lại trường `4G`, là trường có độ lệch không quá cao như các trường còn lại.

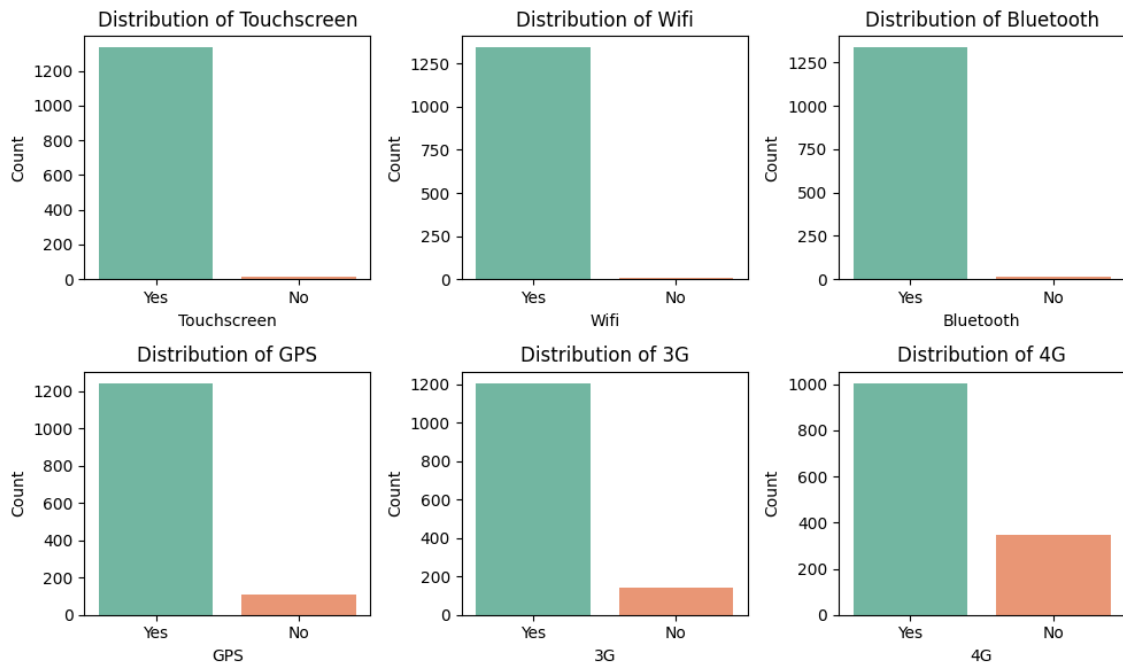


Figure 3: Biểu đồ của một số cột phân loại

Đối với trường *Brand*, là công ty sản xuất ra chiếc điện thoại thì có 75 công ty khác nhau. Nhóm định gom các *Brand* có dưới 13 giá trị lại thành loại *Other*. Sau khi thực hiện thì tính cả *Other* có 32 *brand*. Thực hiện encoding đối với trường này.

Với trường *OS*, 97.77% là *Android*. Với *IOS* thì có giá cao hơn hẳn các giá trị khác. Nên *OS* mang thông tin quan trọng để dự đoán giá bán Nhóm chia *OS* thành *IOS*, *Android* và *Other*. Với trường *Processor*, là số nhân xử lý của vi xử lý trung tâm. Đa phần các thiết bị có bộ xử lý từ **4 đến 8 nhân**, một phần nhỏ có **1 hoặc 2 nhân**. Nhóm đã chia trường *Processor* thành ba khoảng giá trị: **1_2**, **4_6**, và **8_10**

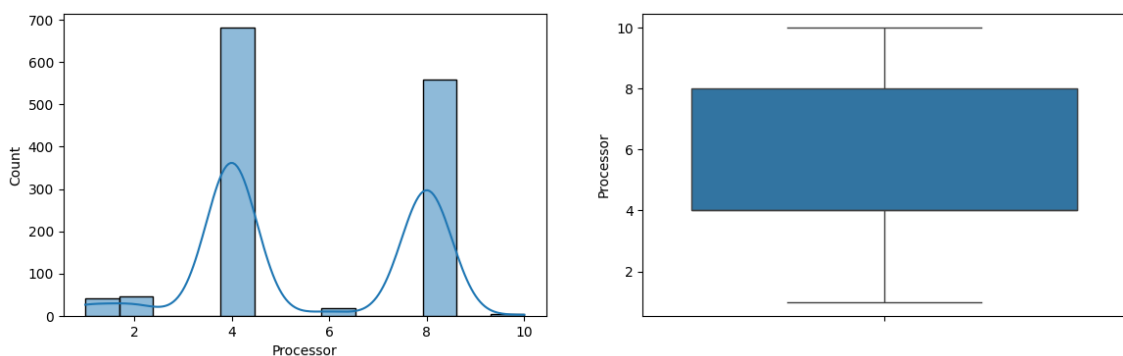


Figure 4: Biểu đồ của một số cột phân loại

Thực hiện Onehot Encoding đối với các trường phân loại. Ta được bộ dữ liệu đã tiền xử lý, tiens hành ghi ra tệp với tên `preprocessed_data.csv`.

III Cơ sở lý thuyết

1 Principal component analysis

PCA (Principal components analysis - Phân tích thành phần chính) đi tìm một phép xoay trục toạ độ để được một hệ trục toạ độ mới sao cho trong hệ mới này, thông tin của dữ liệu chủ yếu tập trung ở một vài thành phần. Phần còn lại chưa ít thông tin hơn có thể được lược bỏ.

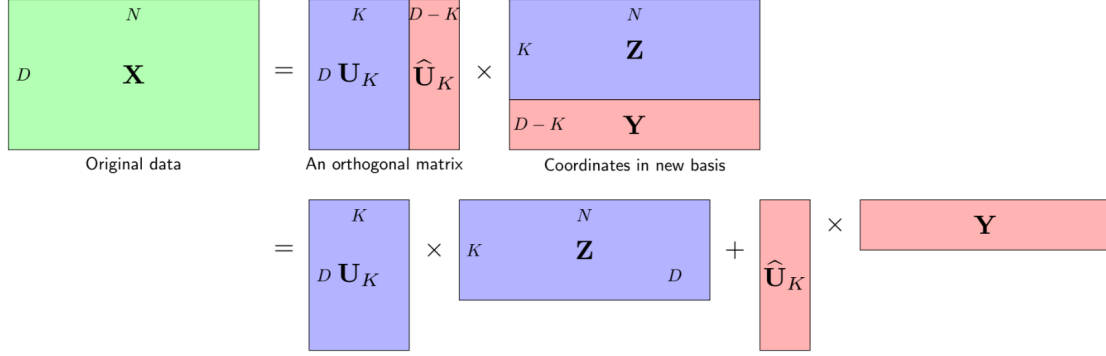


Figure 5: Ý tưởng chính của PCA - Tìm một hệ trục chuẩn mới sao cho trong hệ này, các thành phần quan trọng nhất nằm trong K thành phần đầu tiên.

Giả sử hệ cơ sở trục chuẩn mới là U . Quan sát hình 6, với cơ sở mới $U = [U_K, \hat{U}_K]$ là một hệ trục chuẩn U_k là ma trận con tạo bởi K cột đầu tiên của U .

Theo hệ cơ sở mới, ma trận dữ liệu X có thể viết lại thành

$$X = U_K Z + \hat{U}_K Y \quad (1)$$

Từ đây ta suy ra:

$$\begin{bmatrix} Z \\ Y \end{bmatrix} = \begin{bmatrix} U_K^T \\ \hat{U}_K^T \end{bmatrix} X \Rightarrow \begin{aligned} Z &= U_K^T X \\ Y &= \hat{U}_K^T X \end{aligned} \quad (2)$$

Các bước thực hiện phương pháp PCA có thể tóm tắt như sau:

1. Tính vector kỳ vọng của toàn bộ dữ liệu: $\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$.
2. Trừ mỗi điểm dữ liệu đi vector kỳ vọng của toàn bộ dữ liệu để được dữ liệu chuẩn hoá:

$$\hat{\mathbf{x}}_n = \mathbf{x}_n - \bar{\mathbf{x}} \quad (3)$$

3. Đặt $\hat{X} = [\hat{\mathbf{x}}_1, \hat{\mathbf{x}}_2, \dots, \hat{\mathbf{x}}_D]$ là ma trận dữ liệu chuẩn hoá, tính ma trận hiệp phương sai

$$S = \frac{1}{N} \hat{X} \hat{X}^T \quad (4)$$

4. Tính các trị riêng và vector riêng tương ứng có l_2 norm bằng 1 của ma trận này, sắp xếp chúng theo thứ tự giảm dần của trị riêng.
5. Chọn K vector riêng ứng với K trị riêng lớn nhất để xây dựng ma trận U_K có các cột tạo thành một hệ trực giao. K vectors này, còn được gọi là các thành phần chính, tạo thành một không gian con gần với phân bố của dữ liệu ban đầu đã chuẩn hoá.
6. Chiếu dữ liệu ban đầu đã chuẩn hoá $\hat{X}$ xuống không gian con tìm được.
7. Dữ liệu mới chính là toạ độ của các điểm dữ liệu trên không gian mới: $Z = U_K^T \hat{X}$.

Dữ liệu ban đầu có thể tính được xấp xỉ theo dữ liệu mới bởi $\mathbf{x} \approx U_K \mathbf{z} + \bar{\mathbf{x}}$.

Một điểm dữ liệu mới $\mathbf{v} \in \mathbb{R}^D$ (có thể không nằm trong tập huấn luyện) sẽ được giảm chiều bằng PCA theo công thức $\mathbf{w} = U_K^T (\mathbf{v} - \bar{\mathbf{x}})$. Ngược lại, nếu biết $\mathbf{w}$, ta có thể xấp xỉ $\mathbf{v}$ bởi $U_K \mathbf{w} + \bar{\mathbf{x}}$. Các bước thực hiện PCA được minh hoạ trong hình sau

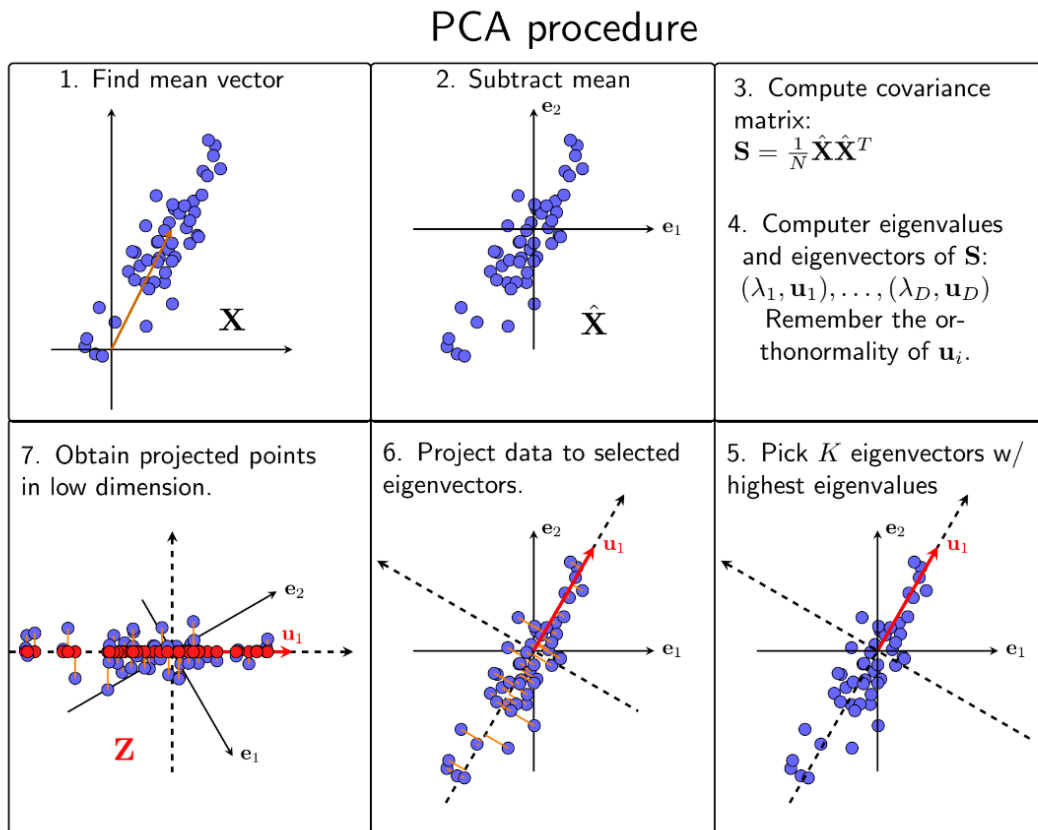


Figure 6: Các bước thực hiện PCA

2 Linear discriminant analysis

Ở phần trên nhóm đã trình bày về principle component analysis (PCA). PCA là một mô hình unsupervised learning, tức là chỉ sử dụng các dữ liệu mà không cần tới nhãn. Tuy

nhân trong bài toán phân lớp, việc khai thác mối quan hệ của dữ liệu và nhãn sẽ mang lại kết quả tốt hơn.

Ví dụ trong bài toán phân lớp nhị phân được mô tả trong hình bên dưới. Ta cần phân loại các điểm dữ liệu thuộc hai lớp xanh và đỏ. Chiếu dữ liệu lên đường thẳng d_1 ứng với phương pháp PCA. Khi đó các điểm dữ liệu chồng lấn lên nhau quá nhiều, khó cho việc phân loại. Nếu chiếu lên đường thẳng d_2 ta nhận thấy các điểm dữ liệu phân tách rõ ràng, thuận lợi hơn cho việc phân loại hai lớp. Như vậy trong bài toán phân loại này, ta cần tìm phương chiếu sao cho các điểm dữ liệu phân tách nhau nhiều nhất có thể, mặc dù lượng thông tin giữ lại không được nhiều nhất.

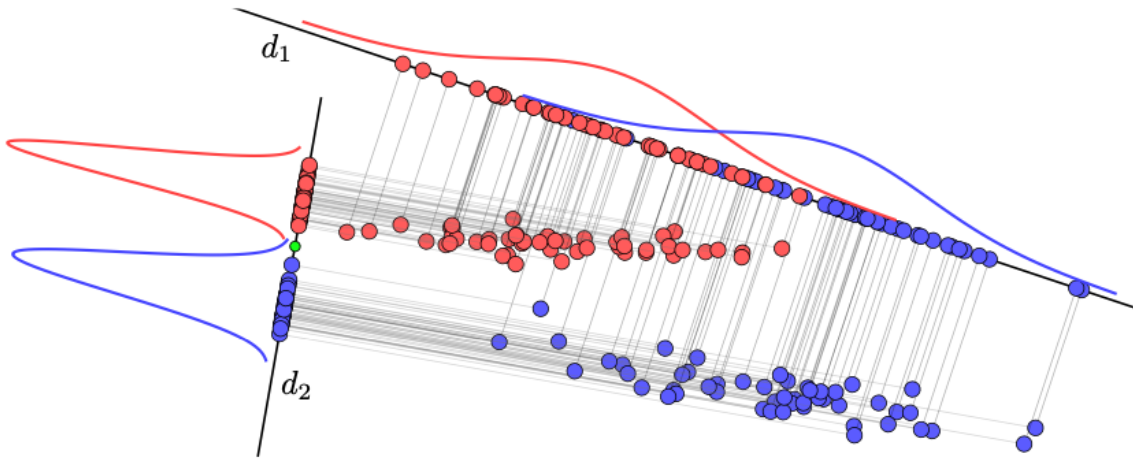


Figure 7: Chiếu dữ liệu lên hai đường thẳng.

Linear discriminant analysis (LDA - Phân tích phân biệt tuyến tính) được ra đời nhằm tìm ra phương chiếu dữ liệu sao cho hiệu quả đối với bài toán phân lớp. Số chiều của dữ liệu mới là nhỏ hơn hoặc bằng $C-1$, trong đó C là số lượng lớp dữ liệu. Từ discriminant được hiểu là lượng thông tin đặc trưng cho mỗi lớp.

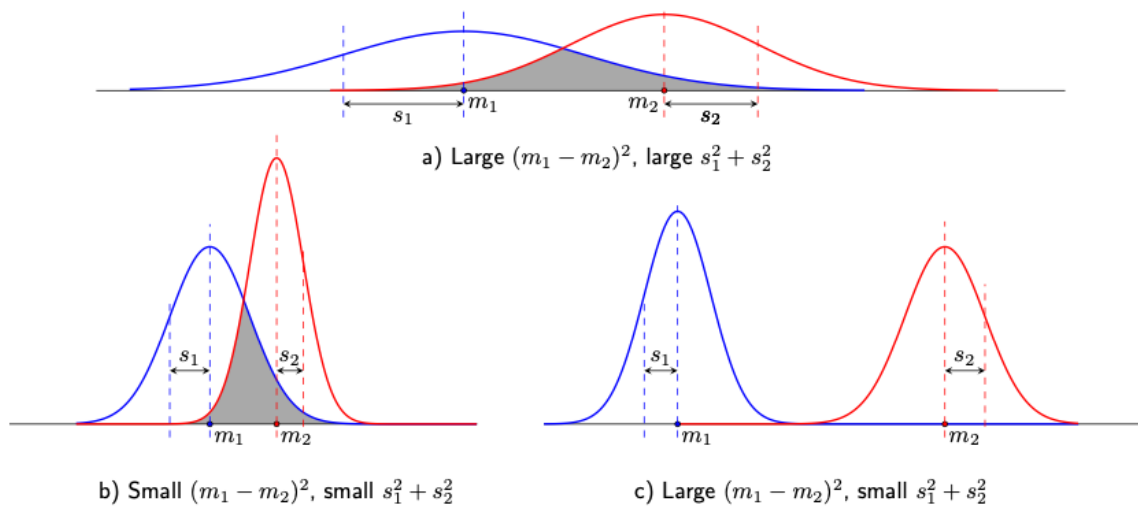


Figure 8: Minh họa dữ liệu sử dụng phân bố chuẩn ứng với các trường hợp

Trong hình minh họa, ta dùng đồ thị phân bố chuẩn(hình chuông) để minh họa. Hình a có khoảng cách giữa hai kỳ vọng là lớn nhưng phương sai trong mỗi lớp cũng lớn khiến cho hai phân phối chồng lấn lên nhau. Hình b, phương sai của mỗi lớp là nhỏ nhưng hai kỳ vọng lại quá gần nhau, khiến cho chồng lấn nhiều, khó phân biệt được hai lớp. Hình c, phương sai đủ nhỏ và khoảng cách giữa hai kỳ vọng đủ lớn, ta thấy dữ liệu sẽ dễ phân biệt các lớp hơn.

Trong phạm vi bài báo cáo này, phần chứng minh toán học cho phương pháp LDA không được trình bày ở đây.

3 Các phương pháp phân cụm

a) Phương pháp K trung bình

Trong K-means clustering, chúng ta không biết nhãn của từng điểm dữ liệu. Mục đích là làm thế nào để phân dữ liệu thành các cụm (cluster) khác nhau sao cho dữ liệu trong cùng một cụm có những tính chất giống nhau.

Mô tả thuật toán phân cụm K-means:

Đầu vào: Ma trận dữ liệu $\mathbf{X} \in \mathbb{R}^{d \times N}$ và số lượng cụm cần tìm $K < N$.

Đầu ra: Ma trận các tâm $\mathbf{M} \in \mathbb{R}^{d \times K}$ và ma trận label $\mathbf{Y} \in \mathbb{R}^{N \times K}$.

1. Chọn K điểm bất kỳ trong training set làm các tâm ban đầu.
2. Phân mỗi điểm dữ liệu vào cụm có tâm gần nó nhất.
3. Nếu việc phân nhóm dữ liệu vào từng cụm ở bước 2 không thay đổi so với vòng lặp trước nó thì ta dừng thuật toán.
4. Cập nhật tâm cho từng cụm bằng cách lấy trung bình cộng của tất cả các các điểm dữ liệu đã được gán vào cụm đó sau bước 2.
5. Quay lại bước 2.

b) Phương pháp DBScan

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) là thuật toán phân cụm dựa trên mật độ. Với phương pháp này, ta không cần số cụm đầu vào như K-means, những vùng có mật độ điểm cao sẽ tạo thành 1 cụm, những vùng thưa điểm sẽ bị xem là nhiễu.

Một số khái niệm cơ bản:

Với tham số:

ϵ : bán kính lân cận, $MinPts$: số điểm tối thiểu để tạo mật độ cao

1. **Điểm lõi (Core Point):** Điểm p là điểm lõi nếu:

$$|N_\epsilon(p)| \geq MinPts$$

2. **Điểm biên (Border Point):** Nằm trong vùng lân cận của điểm lõi nhưng không phải điểm lõi.

3. **Điểm nhiễu (Noise):** Không phải lõi, không phải biên:

$$|N_\epsilon(p)| < MinPts$$

4. **Quan hệ mật độ:**

- Density-Reachable
- Density-Connected

Mô tả thuật toán DBSCAN:

Input: Dữ liệu D , tham số $\epsilon, MinPts$

Pseudo-code:

1. Gán tất cả điểm là chưa được thăm.
2. Với mỗi điểm p :
 - Nếu p đã thăm $\rightarrow$ bỏ qua.
 - Lấy tập lân cận $N_\epsilon(p)$.
 - Nếu $|N_\epsilon(p)| < MinPts$:
 - Gán p là nhiễu.
 - Ngược lại:
 - Tạo cụm mới.
 - Mở rộng cụm bằng cách thêm các điểm density-reachable.

c) Mô hình trộn Gaussian - GMM

Mục tiêu của GMM là ước lượng tham số phù hợp nhất cho C cụm thông qua phương pháp ước lượng hợp lý cực đại MLE.

Thuật toán EM cho GMM:

1. **Khởi tạo:** Chọn số cụm K và khởi tạo các tham số μ_k, Σ_k, π_k .

2. **E-step:** Tính xác suất một điểm x_n thuộc về cụm k :

$$\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n | \mu_j, \Sigma_j)}$$

3. **M-step:** Cập nhật lại các tham số dựa trên xác suất vừa tính:

- $\mu_k^{new} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n$
- $\Sigma_k^{new} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k^{new})(x_n - \mu_k^{new})^T$
- $\pi_k^{new} = \frac{N_k}{N}$

4. **Kiểm tra hội tụ:** Dừng nếu hàm log-likelihood hội tụ, ngược lại quay lại bước 2.

4 K-NN Method

K-nearest neighbor (KNN) là một trong những thuật toán học có huấn luyện đơn giản được xếp vào loại lazy learning, do thuật toán này không học một điều gì từ dữ liệu huấn luyện mà nhớ lại một cách máy móc toàn bộ dữ liệu đó. KNN có thể áp dụng được vào cả bài toán Hồi quy và Phân loại, tuy nhiên ta sẽ chỉ xét với bài toán Hồi quy.

Trong bài toán Hồi quy, đầu ra của một điểm dữ liệu sẽ bằng chính đầu ra của điểm dữ liệu đã biết gần nhất (trong trường hợp $K = 1$), hoặc là trung bình có trọng số của đầu ra của những điểm gần nhất, hoặc bằng một mối quan hệ dựa trên các điểm gần nhất đó và khoảng cách tới chúng.

Sau đây là các bước chi tiết của thuật toán:

Đầu vào

Tập huấn luyện $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, tham số k , hàm khoảng cách.

Chuẩn bị

1. **Làm sạch/tiền xử lý:** xử lý thiếu/ngoại lai; chuẩn hoá.
2. **Chọn k :** dùng k -fold CV; với nhị phân nên chọn k lẻ để tránh hoà phiếu.
3. **Chọn độ đo:** Euclid/Manhattan theo bản chất đặc trưng.

Dự đoán cho một điểm $\mathbf{x}_{new}$:

1. Tính $d_i = d(\mathbf{x}_{new}, \mathbf{x}_i)$ với mọi điểm huấn luyện.
2. Sắp xếp theo d_i tăng dần, lấy $\mathcal{N}_k(\mathbf{x}_{new})$.

3. **Hồi quy:** $\hat{y} = \frac{\sum_{i \in \mathcal{N}_k} w_i y_i}{\sum_{i \in \mathcal{N}_k} w_i}$, mặc định $w_i = 1$.
4. **Ràng buộc thực tế:** cần cân bằng lớp (resampling/class-weight); chuẩn hoá đặc trưng cả train/test.
5. **Đánh giá:** dùng CV/hold-out; tinh chỉnh k , độ đo, trọng số.

Lưu ý:

1. Tính khoảng cách: có 2 loại phổ biến là Euclidean $\|\mathbf{x} - \mathbf{z}\|_2$ (dùng cho dữ liệu nhiều chiều) và Manhattan $\|\mathbf{x} - \mathbf{z}\|_1$ (dùng cho dữ liệu 1 chiều)
2. Chọn giá trị k thỏa mãn: $0 < k < n$ cho bài toán hồi quy.

5 Hồi quy tuyến tính

Lý Thuyết Cơ Bản về Hồi quy Tuyến tính

Mục tiêu của **Hồi quy Tuyến tính (Linear Regression)** là tìm một mối quan hệ **tuyến tính** (một đường thẳng hoặc một siêu phẳng) để mô tả tốt nhất mối quan hệ giữa các biến đầu vào (đặc trưng, X) và một biến đầu ra (mục tiêu, y) liên tục.

Mô hình toán học

Giả sử chúng ta có N điểm dữ liệu và D đặc trưng. Mô hình dự đoán $\hat{y}$ cho một điểm dữ liệu $\mathbf{x} = [x_1, x_2, \dots, x_D]$ được biểu diễn là:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_D x_D$$

Trong đó:

- w_0 là **hệ số chặn (intercept)** hay còn gọi là **bias**.
- $w_1, \dots, w_D$ là các **trọng số (weights)** hoặc **hệ số (coefficients)** của mô hình, biểu thị tầm quan trọng của mỗi đặc trưng.

Để tính toán hiệu quả, chúng ta biểu diễn mô hình dưới dạng ma trận. Ta "mở rộng" vector $\mathbf{x}$ bằng cách thêm $x_0 = 1$ (để xử lý w_0). Đặt $\mathbf{x}_b = [1, x_1, \dots, x_D]^T$ và $\mathbf{w} = [w_0, w_1, \dots, w_D]^T$.

Khi đó, với toàn bộ N điểm dữ liệu, ta có ma trận X_b (kích thước $N \times (D + 1)$) và vector y (kích thước $N \times 1$):

$$\hat{y} = X_b \mathbf{w}$$

Mục tiêu của chúng ta là tìm vector trọng số $\mathbf{w}$ sao cho $\hat{y}$ xấp xỉ y nhất có thể. Chúng ta đo lường "xấp xỉ" bằng cách **tối thiểu hóa tổng lỗi bình phương (Sum of Squared**

Errors - SSE):

$$J(\mathbf{w}) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \|y - X_b \mathbf{w}\|_2^2$$

Khai triển QR (QR Decomposition)

Đây là phương pháp **ổn định hơn về mặt số học** và thường được ưu tiên sử dụng trong các thư viện (như `scikit-learn`).

- **Lý thuyết:** Bất kỳ ma trận A (ở đây là X_b) nào cũng có thể được phân rã thành $A = QR$, trong đó:

- Q là một **ma trận trực giao** (các cột vuông góc với nhau và có độ dài bằng 1). Tính chất quan trọng: $Q^T Q = I$ (ma trận đơn vị).
- R là một **ma trận tam giác trên**.

MSE (Mean Squared Error - Lỗi Bình phương Trung bình)

Ý nghĩa: Đo lường trung bình của *bình phương* các sai số. Thước đo này **trừng phạt rất nặng** các sai số lớn (vì chúng được bình phương lên).

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

MAE (Mean Absolute Error - Lỗi Tuyệt đối Trung bình)

Ý nghĩa: Đo lường trung bình của *giá trị tuyệt đối* các sai số. Thước đo này **đễ diễn giải** vì có cùng đơn vị với biến mục tiêu (ví dụ: sai số trung bình là 10 USD).

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

R^2 (R-squared - Hệ số Xác định)

Ý nghĩa: Cho biết *phần trăm (%)* sự biến động của biến mục tiêu (y) được **giải thích** bởi mô hình. R^2 càng gần 1, mô hình càng phù hợp.

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

6 Các bài toán phân loại

a) Phương pháp K-NN cho bài toán phân loại nhiều lớp

Các bước thực hiện phương pháp KNN:

1. Với mỗi x – unseen ở đầu vào (các phần tử trong tập Validation)

2. Tính khoảng cách d_i từ x đến các x_i trong tập Training $\Rightarrow$ Lập thành 1 mảng.
3. Sắp xếp các khoảng cách này theo thứ tự tăng dần, nhưng cần có tham chiếu để biết chỉ số phần tử ban đầu trong mảng d_i
4. Tìm ra K chỉ số của các phần tử x_i trong tập Training ứng với các d_i nhỏ nhất. Gọi tập này là $n_j, j = 1, 2, \dots, K$.
5. Xác định nhãn L có tần suất xuất hiện nhiều nhất trong tập đầu ra của các mẫu huấn luyện y_{n_j} . Đầu ra dự đoán $y_{pred} := L$.

b) Gaussian Naive Bayes (GNB)

Phương pháp này áp dụng cho dữ liệu liên tục, giả định rằng các đặc trưng tuân theo phân phối chuẩn.

1. Bước 1: Chuẩn bị dữ liệu (Data Preparation)

Dữ liệu đầu vào là ma trận số thực. Không yêu cầu rời rạc hóa hay nhị phân hóa.

2. Bước 2: Huấn luyện (Training - Fitting)

Với mỗi lớp c và mỗi đặc trưng i , thuật toán tính toán hai tham số thống kê:

- Trung bình mẫu: μ_{ci}
- Phương sai mẫu: σ_{ci}^2

3. Bước 3: Dự đoán (Prediction)

Xác suất có điều kiện được tính bằng hàm mật độ xác suất (Probability Density Function - PDF) của phân phối chuẩn:

$$P(x_i|y = c) = \frac{1}{\sqrt{2\pi\sigma_{ci}^2}} \exp\left(-\frac{(x_i - \mu_{ci})^2}{2\sigma_{ci}^2}\right)$$

Sau đó chọn lớp có xác suất hậu nghiệm lớn nhất.

c) Bernoulli Naive Bayes (BNB)

Phương pháp này áp dụng cho dữ liệu nhị phân (0/1), mô hình hóa sự hiện diện hoặc vắng mặt của đặc trưng.

1. Bước 1: Nhị phân hóa dữ liệu (Binarization)

Nếu dữ liệu chưa phải dạng 0/1, cần xác định một ngưỡng (threshold) θ .

$$x'_i = \begin{cases} 1 & \text{nếu } x_i > \theta \\ 0 & \text{nếu } x_i \leq \theta \end{cases}$$

2. Bước 2: Huấn luyện (Training)

Tính toán xác suất xuất hiện của đặc trưng i trong lớp c , sử dụng kỹ thuật làm trơn Laplace (Laplace smoothing) để tránh lỗi xác suất bằng 0:

$$P(x_i = 1|y = c) = \frac{N_{ci} + \alpha}{N_c + 2\alpha}$$

Trong đó N_{ci} là số mẫu thuộc lớp c có đặc trưng $i = 1$, và N_c là tổng số mẫu thuộc lớp c .

3. Bước 3: Dự đoán (Prediction)

Sử dụng công thức phân phối Bernoulli, trừng phạt (penalize) sự vắng mặt của đặc trưng đặc trưng quan trọng:

$$P(x_i|y = c) = P(i|c) \cdot x_i + (1 - P(i|c)) \cdot (1 - x_i)$$

d) Phương pháp Hồi quy SoftMax

Hồi quy Softmax là mô hình tổng quát hóa của hồi quy Logistic cho bài toán phân loại đa lớp ($K > 2$). Quy trình thực hiện bao gồm các bước sau:

1. Tính điểm số tuyến tính (Linear Logits):

Với vector đầu vào $\mathbf{x} \in \mathbb{R}^d$ và K lớp cần phân loại. Mỗi lớp k ($k = 1 \dots K$) có một vector trọng số $\mathbf{w}_k$ và bias b_k . Điểm số (score) z_k của lớp k được tính:

$$z_k = \mathbf{w}_k^T \mathbf{x} + b_k$$

2. Tính xác suất bằng hàm Softmax:

Chuyển đổi các điểm số z thành phân phối xác suất $\hat{y}$. Xác suất để đầu vào $\mathbf{x}$ thuộc lớp k là:

$$\hat{y}_k = P(y = k|\mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Hàm này đảm bảo $\hat{y}_k \in (0, 1)$ và $\sum_{k=1}^K \hat{y}_k = 1$.

3. Hàm mất mát (Cross-Entropy Loss):

Để đánh giá sai số giữa dự đoán $\hat{y}$ và nhãn thực tế y (đã được mã hóa one-hot), ta sử dụng hàm Cross-Entropy:

$$J(\mathbf{W}) = - \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik})$$

Trong đó N là số lượng mẫu dữ liệu.

4. Tối ưu hóa tham số:

Sử dụng thuật toán Gradient Descent (batch hoặc stochastic) để cập nhật ma trận trọng số $\mathbf{W}$ nhằm cực tiểu hóa hàm mất mát $J(\mathbf{W})$:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} J(\mathbf{W})$$

(Với η là tốc độ học - learning rate).

5. Dự đoán:

Nhân dự đoán cho một mẫu mới là lớp có xác suất cao nhất:

$$\text{class} = \arg \max_k (\hat{y}_k)$$

7 Validation

a) Validation

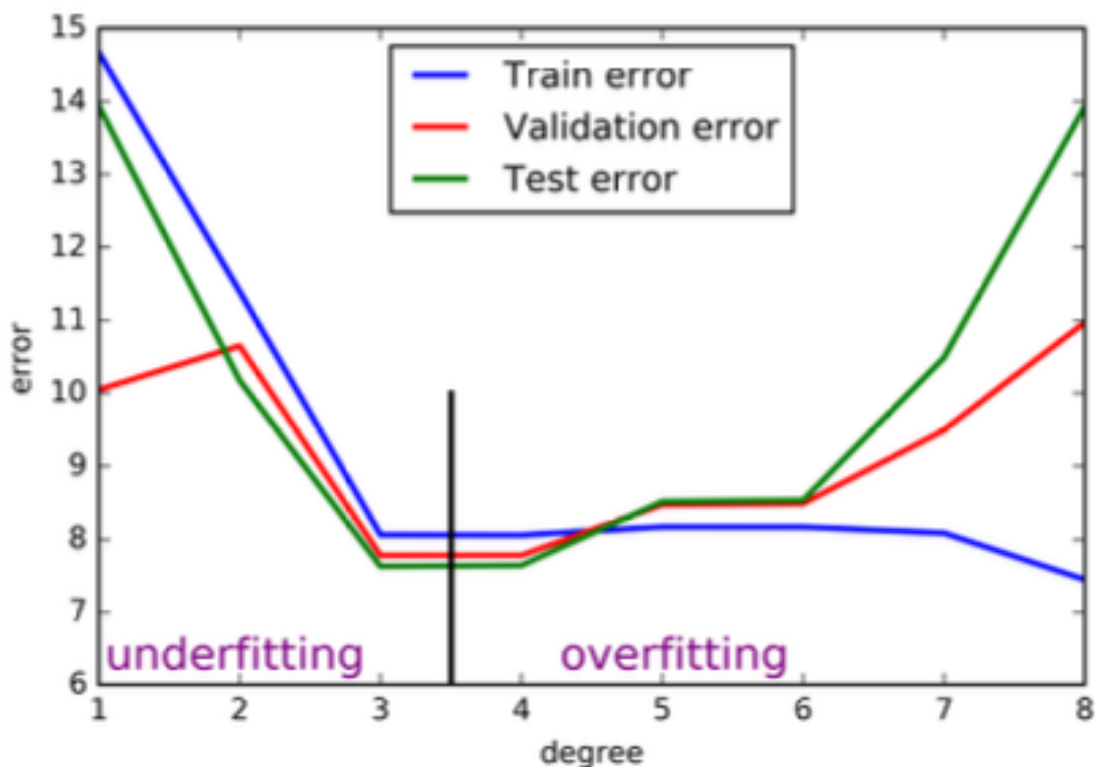


Figure 9: Lựa chọn mô hình dựa trên validation error.

Một mô hình là tốt nếu cả training error và test error đều nhỏ. Tuy nhiên, khi xây dựng một mô hình chỉ dựa trên training set, làm thế nào để biết được chất lượng của nó trên test set?

Phương pháp đơn giản nhất là trích từ training set ra một tập con nhỏ và thực hiện việc đánh giá mô hình trên tập con nhỏ này. Tập con nhỏ được trích ra từ training set này được gọi là validation set. Lúc này, training set mới là phần còn lại của training set ban đầu. Việc này khá giống với việc chúng ta ôn thi. Giả sử các đề thi của các năm trước là training set, đề thi năm nay là test set mà ta chưa biết. Khi ôn tập, ta thường chia đề các năm trước ra hai phần: phần thứ nhất có thể xem lời giải và tài liệu để ôn tập, phần còn lại ta tự làm mà không sử dụng tài liệu để tự đánh giá kiến thức của mình. Lúc này, phần thứ nhất đóng vai trò là training set mới, trong khi phần thứ hai chính là validation set. Nếu kết quả bài làm trên phần thứ hai là khả quan, ta có thể tự tin hơn khi vào bài thi thật.

Lúc này, có ba đại lượng cần được quan tâm: training error trên training set mới, validation error được định nghĩa tương tự trên validation set, và test error trên test set. Với khái niệm mới này, ta tìm mô hình sao cho cả train error và validation error đều nhỏ, qua đó có thể dự đoán được rằng test error cũng nhỏ. Để làm được việc này, ta có thể huấn luyện nhiều mô hình khác nhau dựa trên training set, sau đó áp dụng các mô hình tìm được và tính validation error. Mô hình cho validation error nhỏ nhất sẽ là một mô hình tốt.

Thông thường, ta bắt đầu từ mô hình đơn giản, sau đó tăng dần độ phức tạp của mô hình. Khi độ phức tạp này tăng lên, training error sẽ có xu hướng nhỏ dần, nhưng điều tương tự có thể không xảy ra ở validation error. Validation error ban đầu thường giảm dần và đến một lúc sẽ tăng lên do overfitting xảy ra trên training khi độ phức tạp của mô hình tăng lên. Để chọn ra một mô hình tốt, ta quan sát validation error. Khi validation error có chiều hướng tăng lên thì ta chọn mô hình trước đó.

Hình trên mô tả ví dụ phía trên với bậc của đa thức tăng từ một đến tám. Validation set là 10 điểm được lấy ra ngẫu nhiên từ training set 30 điểm ban đầu. Chúng ta hãy tạm chỉ xét hai đường màu lam và đỏ, tương ứng với training error và validation error. Khi bậc của đa thức tăng lên, training error có xu hướng giảm. Điều này dễ hiểu vì đa thức bậc càng cao, việc xấp xỉ càng chính xác. Quan sát đường màu đỏ, khi bậc của đa thức là ba hoặc bốn thì validation error thấp, sau đó nó tăng dần lên. Dựa vào validation error, ta có thể xác định được bậc cần chọn là ba hoặc bốn. Quan sát tiếp đường màu lục, tương ứng với test error, thật là trùng hợp, với bậc bằng ba hoặc bốn, test error cũng đạt giá trị nhỏ nhất, sau đó tăng dần lên. Vậy cách làm này ở đây đã tỏ ra hiệu quả. Và mô hình phù hợp là mô hình có bậc bằng ba hoặc bốn. Trong ví dụ này, validation set đóng vai trò tìm ra bậc của đa thức, training set đóng vai trò trong việc tìm các hệ số của đa thức với bậc đã biết. Các hệ số của đa thức chính là các model parameter, trong khi bậc của đa thức có thể được coi là hyperparameter. Cả training set và validation set đều đóng vai trò xây dựng mô hình. Nhắc lại rằng hai tập hợp này được tách ra từ training set ban đầu.

Việc không sử dụng test data khi lựa chọn mô hình ở trên nhưng vẫn có được kết quả

khả quan vì ta đã giả sử rằng validation data và test data có chung một đặc điểm nào đó (chung phân phối và đều chưa được mô hình nhìn thấy khi huấn luyện). Và khi cả hai đều là chưa được nhìn thấy, error trên hai tập này sẽ tương đối giống nhau.

b) Cross-validation

Trong nhiều trường hợp, chúng ta có rất hạn chế số lượng dữ liệu để xây dựng mô hình. Nếu lấy quá nhiều dữ liệu trong training set ra làm dữ liệu validation, phần dữ liệu còn lại của training set là không đủ để xây dựng mô hình. Lúc này, validation set phải thật nhỏ để giữ được lượng dữ liệu cho training đủ lớn. Tuy nhiên, một vấn đề khác nảy sinh. Khi validation set quá nhỏ, hiện tượng overfitting lại có thể xảy ra với training set còn lại. Có giải pháp nào cho tình huống này không? Câu trả lời là *cross-validation*.

Cross-validation là một cải tiến của phương pháp *validation*, trong đó tập dữ liệu dùng để đánh giá (validation set) có kích thước nhỏ nhưng mô hình được kiểm định trên nhiều tập dữ liệu khác nhau, nhằm đảm bảo tính tổng quát và độ tin cậy của kết quả đánh giá.

Một cách thường được sử dụng là chia *training set* thành k tập con không giao nhau, có kích thước gần bằng nhau. Ở mỗi lần lặp (gọi là một *run*), một trong số k tập con được lấy ra làm *validation set*, trong khi hợp của $k - 1$ tập còn lại được dùng để huấn luyện mô hình.

Sau k lần lặp, ta thu được k mô hình khác nhau. Mô hình cuối cùng được xác định dựa trên trung bình của *training error* và *validation error* qua các lần chạy. Cách làm này được gọi là ***k-fold cross-validation***.

Khi k bằng với số lượng phần tử trong tập huấn luyện ban đầu, tức là mỗi tập con chỉ có đúng một phần tử, kỹ thuật này được gọi là ***leave-one-out cross-validation***.

IV Phân tích và trực quan hóa dữ liệu

Dữ liệu sau khi được nhóm tiền xử lý, được phân tích thống kê. Phân tích thống kê của các trường dạng số được thể hiện trong hình bên dưới.

	count	mean	std	min	25%	50%	75%	max
Screen size	1349.0	5.283640	0.665961	2.400	5.0	5.2	5.67	7.0
Resolution x	1349.0	808.260934	268.198554	240.000	720.0	720.0	1080.00	2160.0
Resolution y	1349.0	1483.480356	550.919076	320.000	1280.0	1280.0	1920.00	3840.0
Storage	1349.0	29.814648	33.419657	0.064	8.0	16.0	32.00	256.0
Rear camera	1349.0	11.993625	8.590219	0.000	8.0	12.2	13.00	64.0
Front camera	1349.0	6.998962	6.249636	0.000	2.0	5.0	8.00	48.0
RAM	1349.0	2.458302	1.597354	0.064	1.0	2.0	3.00	12.0
Battery capacity	1349.0	2932.180875	871.867999	1010.000	2300.0	2942.0	3500.00	6000.0

Figure 10: Các tham số thống kê cơ bản

Bảng thống kê mô tả cho thấy các biến liên tục trong tập dữ liệu có sự khác biệt đáng kể về giá trị và mức độ phân tán. Kích thước màn hình (*Screen size*) dao động từ 2.4 đến 7.0 inch, trung bình khoảng 5.28 inch — tương ứng với các dòng điện thoại tầm trung phổ biến. Độ phân giải màn hình theo chiều ngang và dọc (*Resolution x*, *Resolution y*) có độ lệch chuẩn cao, cho thấy sự đa dạng giữa các thiết bị giá rẻ và cao cấp.

Bộ nhớ trong (*Storage*) có giá trị trung bình khoảng 29.8 GB nhưng phân tán mạnh (độ lệch chuẩn 33.4), với một số thiết bị có dung lượng lên đến 256 GB. Độ phân giải camera sau và trước (*Rear camera*, *Front camera*) có nhiều giá trị bằng 0, thể hiện một số mẫu không có camera hoặc thiếu dữ liệu.

Bộ nhớ RAM (*RAM*) dao động từ 0.064 GB đến 12 GB, trung bình khoảng 2.46 GB, phản ánh chênh lệch giữa các thiết bị đời cũ và mới. Dung lượng pin (*Battery capacity*) trung bình khoảng 2932 mAh, chủ yếu nằm trong khoảng 2300–3500 mAh, cho thấy sự phân bố hợp lý quanh mức pin phổ biến của điện thoại thông minh.

Đối với các biến nhị phân (sau khi đã thực hiện one hot encoding), tần suất xuất hiện được thể hiện bên dưới.

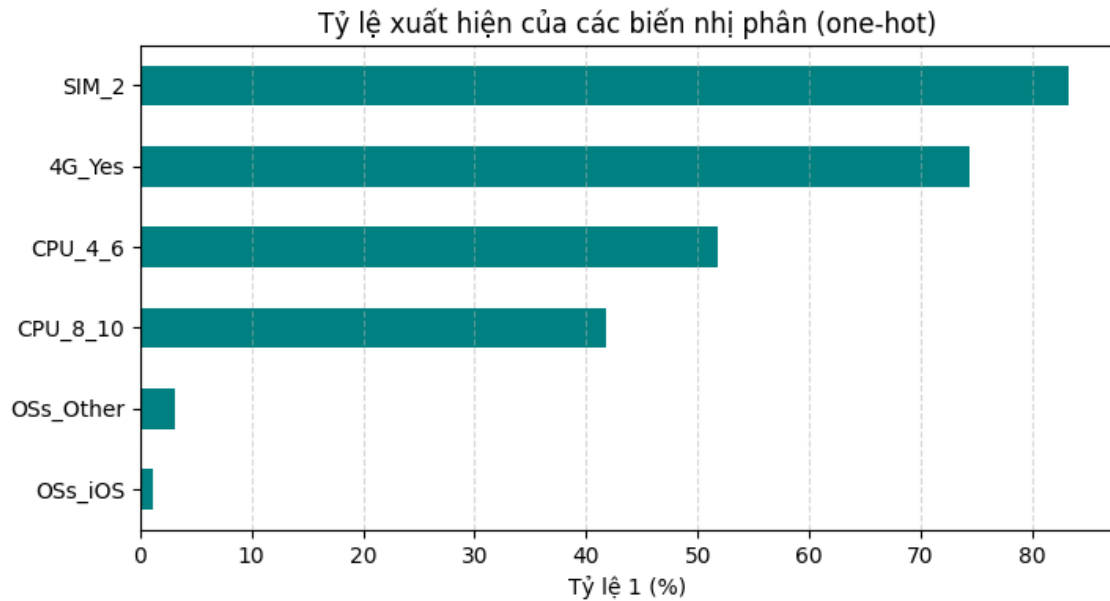


Figure 11: Tỷ lệ số lượng các biến nhị phân

Biểu đồ cho thấy sự phân bố của các biến nhị phân sau khi mã hóa one-hot. Biến SIM_2 và 4G_Yes có tỷ lệ xuất hiện cao nhất, lần lượt trên 80% và 70%, cho thấy phần lớn các thiết bị trong tập dữ liệu hỗ trợ hai SIM và kết nối 4G. Trong khi đó, các biến liên quan đến hệ điều hành như OSs_iOS và OSs_Other chiếm tỷ lệ rất nhỏ, phản ánh rằng đa số điện thoại trong tập dữ liệu sử dụng hệ điều hành Android. Ngoài ra, các nhóm biến CPU_4_6 và CPU_8_10 có tỷ lệ xuất hiện trung bình (khoảng 50%), thể hiện sự phân bố tương đối cân bằng giữa các dòng CPU tầm trung và cao cấp.

Trước khi huấn luyện mô hình, nhóm thực hiện **chuẩn hóa dữ liệu** bằng phương pháp *StandardScaler* nhằm đưa các đặc trưng về cùng thang đo, loại bỏ ảnh hưởng của đơn vị đo lường khác nhau.

1 Phân tích và trực quan với PCA

Sau đó, áp dụng **Principal Component Analysis (PCA)** để *phân tích các thành phần chính*, giảm số chiều dữ liệu đồng thời giữ lại phần lớn phương sai, giúp mô hình học hiệu quả và ổn định hơn.

Biểu đồ phân tán các thành phần chính (PCA)

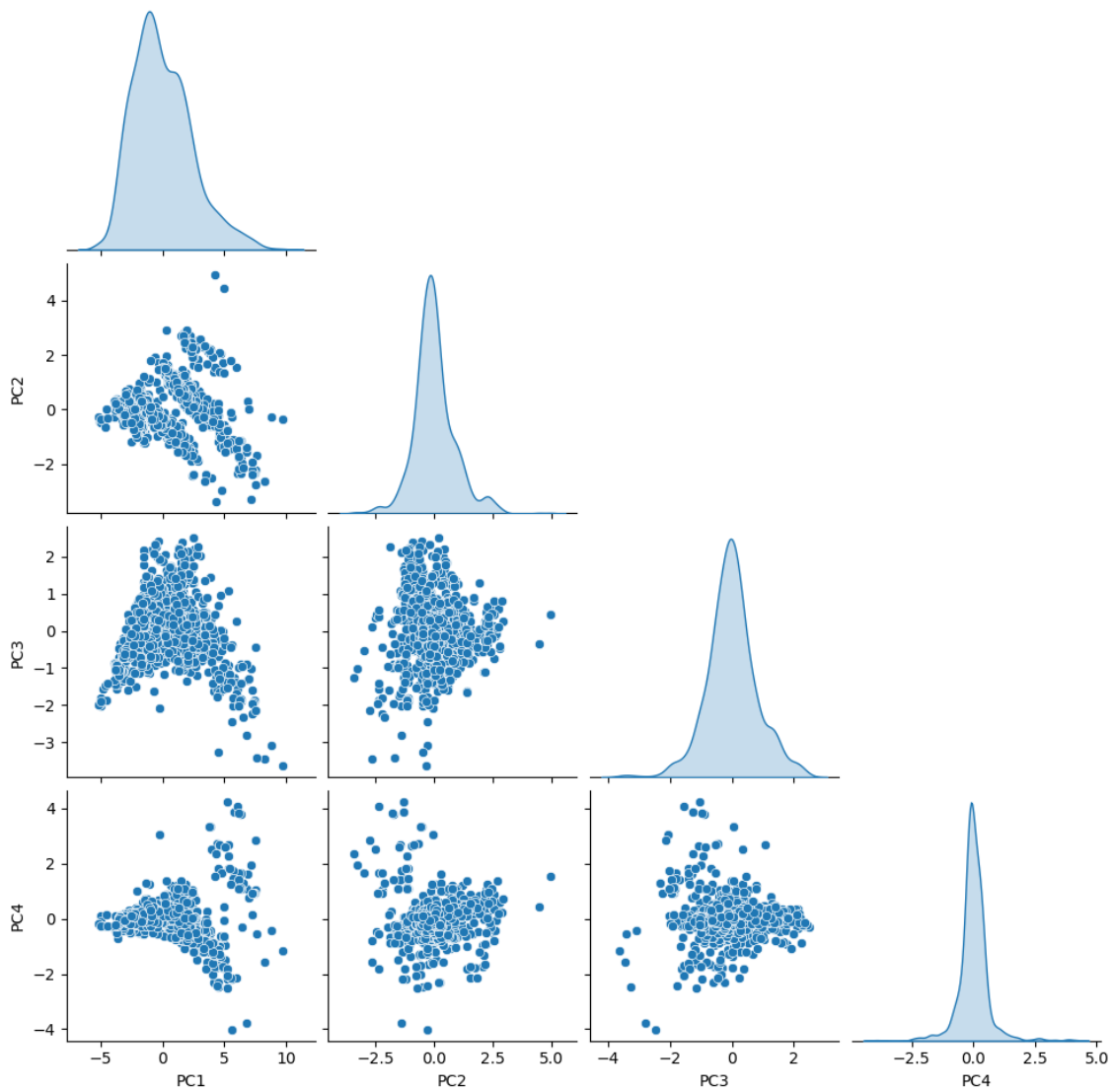


Figure 12: Trực quan theo từng cặp thành phần chính

Biểu đồ thể hiện sự phân tán của dữ liệu theo từng cặp thành phần chính sau khi áp dụng PCA. Các điểm dữ liệu được phân bố tương đối đồng đều, không xuất hiện cụm rõ rệt, cho thấy rằng các thành phần chính (PC1–PC4) đã nắm bắt được phần lớn phương sai trong dữ liệu mà không có sự chồng chéo mạnh giữa các chiều.

Phân phối của từng thành phần (trên đường chéo) có dạng gần chuẩn, phản ánh hiệu quả của quá trình chuẩn hóa và trích xuất thành phần tuyến tính. Điều này cho thấy PCA đã giúp dữ liệu trở nên dễ quan sát và giảm tương quan giữa các biến đầu vào.

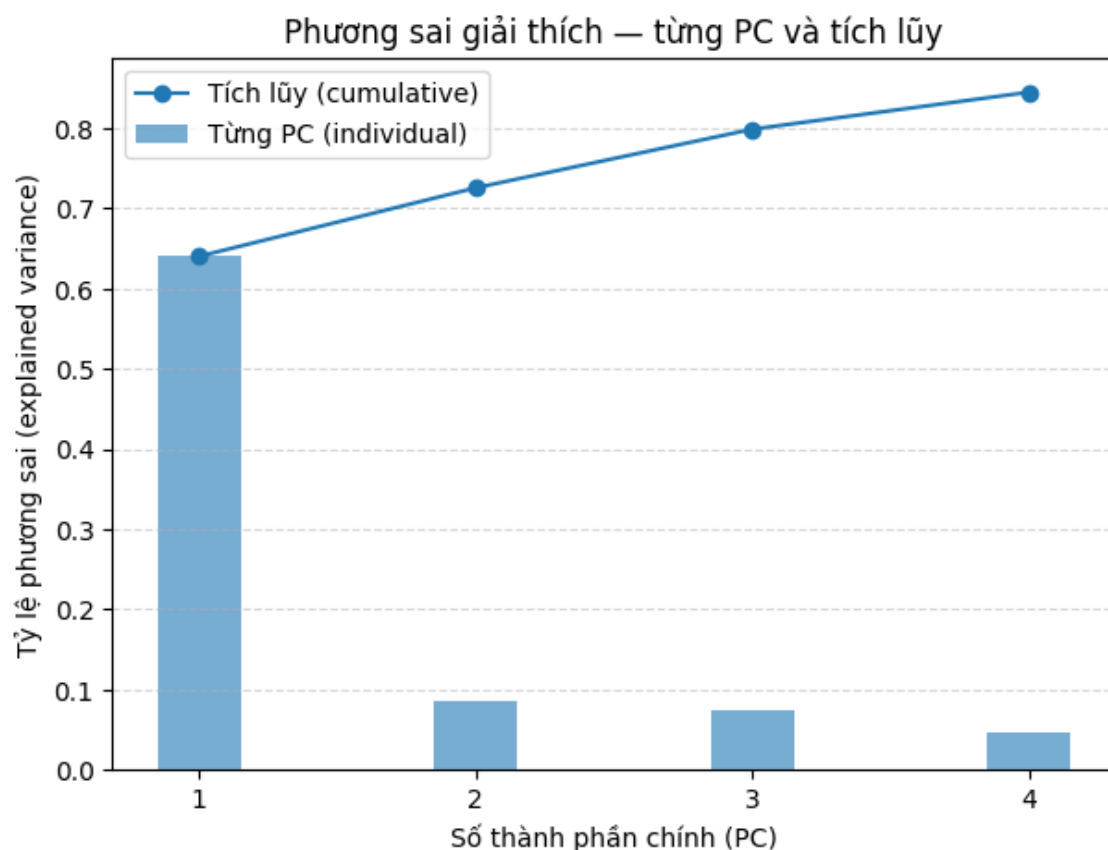


Figure 13: Phương sai giải thích các PC

Biểu đồ thể hiện tỷ lệ phương sai được giải thích bởi từng thành phần chính (PC) và phần tích lũy. Thành phần chính thứ nhất (PC1) chiếm tỷ lệ phương sai cao nhất, khoảng 65%, cho thấy đây là chiều dữ liệu mang nhiều thông tin nhất. Bốn thành phần đầu tiên cộng lại giải thích được hơn 85% tổng phương sai, chứng tỏ phần lớn thông tin ban đầu đã được bảo toàn sau khi giảm chiều.

Do đó, việc lựa chọn 3–4 thành phần chính là hợp lý, vừa đảm bảo giữ lại phần lớn thông tin của dữ liệu, vừa giúp mô hình đơn giản và hiệu quả hơn.

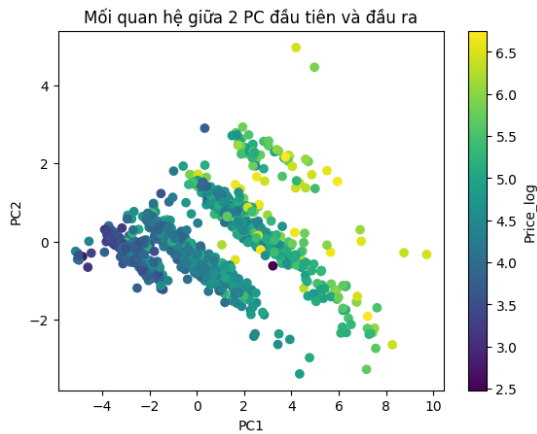


Figure 14: Mối quan hệ giữa PC1, PC2 và đầu ra

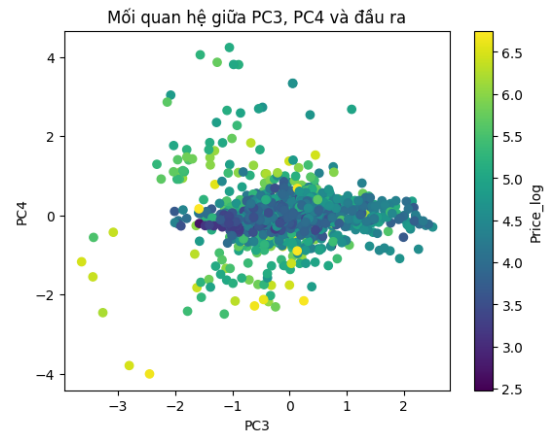


Figure 15: Mối quan hệ giữa PC3, PC4 và đầu ra

Hai biểu đồ thể hiện mối quan hệ giữa các thành phần chính (PC1–PC2, PC3–PC4) và biến đầu ra `Price_log`. Quan sát cho thấy các điểm dữ liệu có xu hướng thay đổi dọc theo chiều PC1, trong khi PC2, PC3 và PC4 không thể hiện mối tương quan tuyến tính rõ rệt với đầu ra.

Điều này cho thấy thành phần chính đầu tiên (PC1) đóng vai trò quan trọng nhất trong việc giải thích sự biến thiên của giá điện thoại, còn các thành phần sau chủ yếu nắm bắt nhiễu hoặc phương sai nhỏ hơn. Do đó, việc giữ lại một số thành phần chính đầu tiên là hợp lý để vừa giảm chiều dữ liệu, vừa bảo toàn thông tin có ý nghĩa với mô hình hồi quy.

2 Phân tích và trực quan với LDA

Nhóm cũng đã tiến hành phân tích trực quan bằng phương pháp LDA. Kết quả được thể hiện cụ thể như sau.

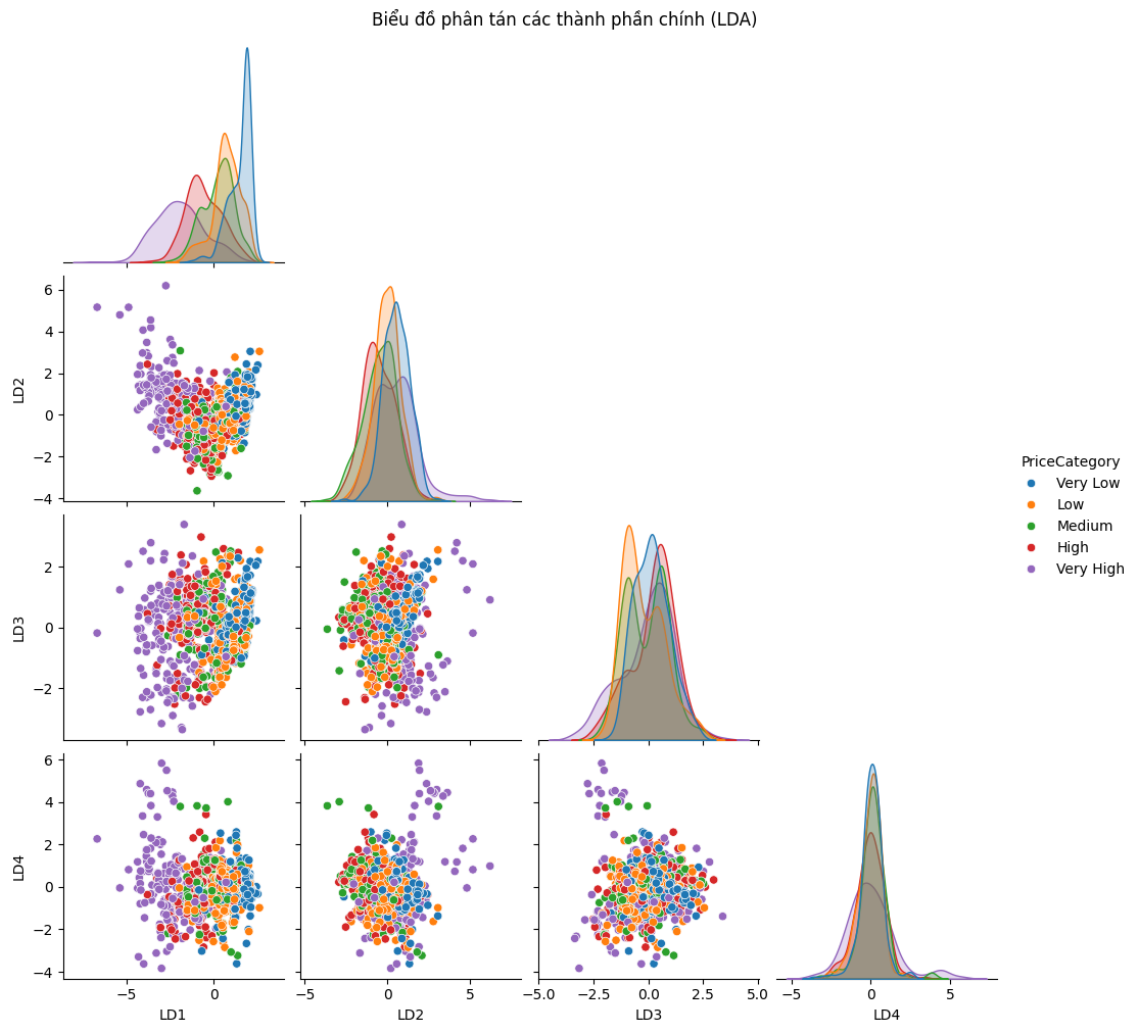


Figure 16: Biểu đồ phân tán các thành phần chính LDA

Biểu đồ phân tán giữa các thành phần LDA (LD1–LD4) cho thấy dữ liệu sau khi chiếu bằng LDA đã thể hiện được sự tách biệt tương đối giữa các nhóm *PriceCategory*. Trong đó, trục LD1 thể hiện khả năng phân biệt rõ nhất giữa các mức giá — các nhóm *Very Low*, *Low*, và *High* có xu hướng hình thành các cụm riêng biệt, dù vẫn tồn tại một số vùng giao nhau giữa *Medium* và *High*. Các thành phần LD2–LD4 đóng vai trò bổ trợ, giúp tinh chỉnh ranh giới giữa các lớp nhưng không thể hiện sự phân tách rõ ràng như LD1. Kết quả này cho thấy LDA đã thành công trong việc trích xuất các đặc trưng có khả năng phân biệt giá trị sản phẩm, đồng thời giúp giảm chiều dữ liệu hiệu quả.

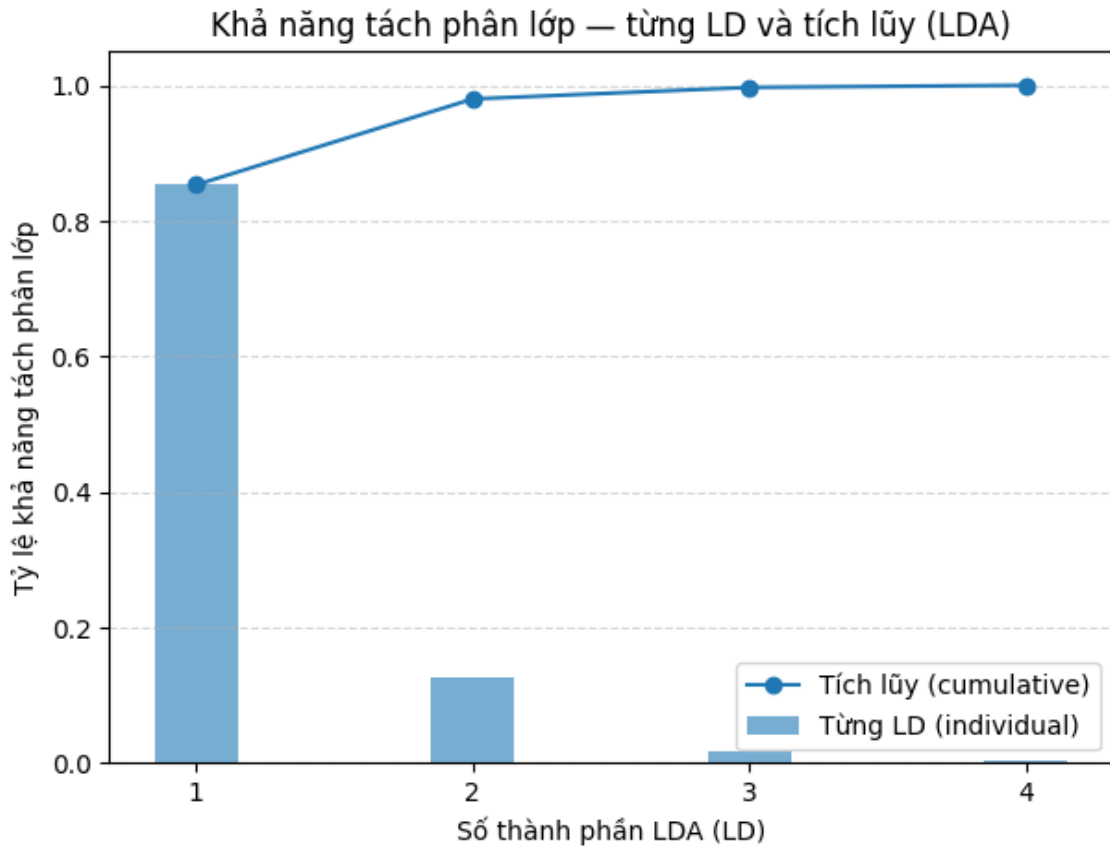


Figure 17: Khả năng tách phân lớp từng LD và tích lũy

Biểu đồ cho thấy thành phần phân biệt đầu tiên (LD1) chiếm tỷ lệ khả năng tách lớn nhất, với giá trị xấp xỉ 85%, thể hiện vai trò chủ đạo trong việc phân tách các nhóm dữ liệu. Thành phần thứ hai (LD2) chỉ đóng góp khoảng 10%, trong khi các thành phần còn lại (LD3, LD4) gần như không mang thêm thông tin đáng kể. Tổng cộng, hai thành phần đầu tiên đã bảo tồn gần 95% lượng thông tin phục vụ cho việc tách lớp, chứng tỏ dữ liệu có thể được biểu diễn hiệu quả trong không gian 2 chiều LDA mà vẫn duy trì gần như toàn bộ khả năng phân biệt giữa các nhóm giá.

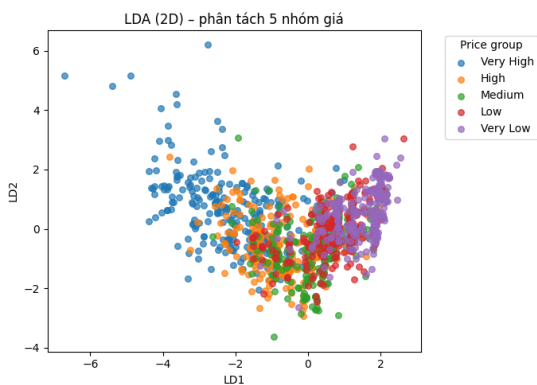


Figure 18: Mối quan hệ giữa LD1, LD2 và đầu ra phân lớp

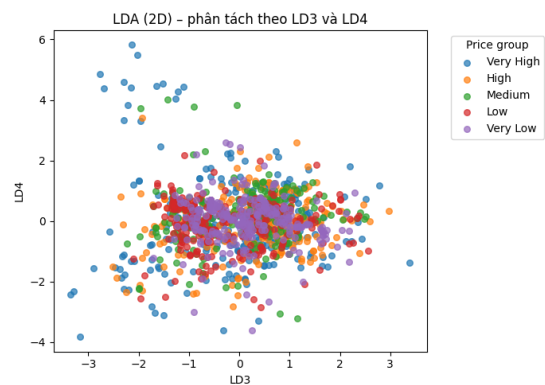


Figure 19: Mối quan hệ giữa LD3, LD4 và đầu ra phân lớp

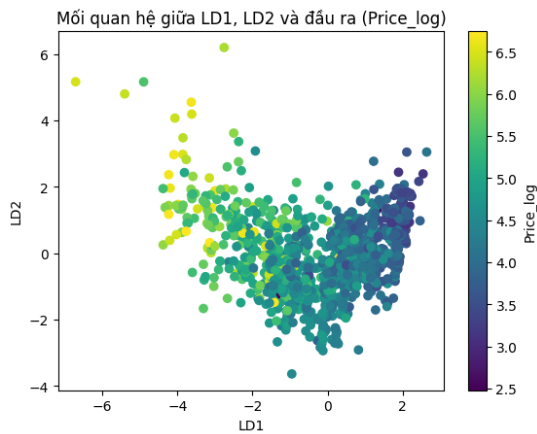


Figure 20: Mối quan hệ giữa LD1, LD2 và đầu ra price_log

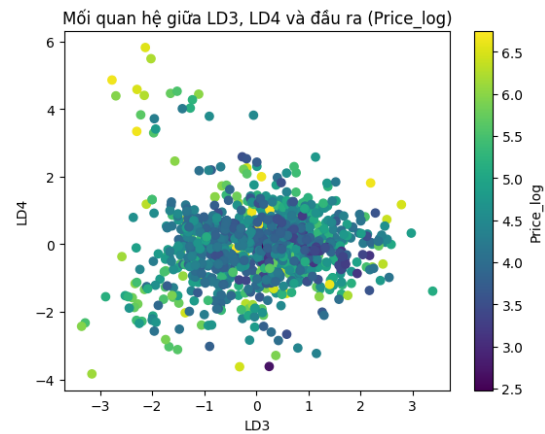


Figure 21: Mối quan hệ giữa LD3, LD4 và đầu ra price_log

Biểu đồ LD1–LD2 cho thấy dữ liệu được phân tách khá rõ ràng giữa các nhóm giá khác nhau (Very Low, Low, Medium, High, Very High). Đặc biệt, các cụm giá thấp (Very Low và Low) nằm về một phía của trục LD1, trong khi các nhóm giá cao hơn có xu hướng dịch chuyển ngược lại. Điều này cho thấy hai thành phần đầu tiên của LDA nắm giữ phần lớn khả năng phân biệt giữa các lớp giá trị.

Ngược lại, biểu đồ LD3–LD4 thể hiện sự chồng lấn lớn giữa các nhóm, cho thấy hai thành phần này chỉ đóng góp nhỏ vào việc tách lớp — phù hợp với kết quả phương sai giải thích, khi LD1 chiếm hơn 80% khả năng phân tách.

Khi xem xét mối quan hệ giữa các thành phần LDA và giá trị đầu ra thực tế (Price_log), các biểu đồ LD1–LD2 cho thấy xu hướng tuyến tính rõ ràng: giá trị Price_log tăng dần theo chiều giảm của LD1, thể hiện rằng thành phần đầu tiên của LDA không chỉ hữu ích cho phân loại mà còn có tương quan mạnh với giá trị dự đoán. Trong khi đó, các thành phần LD3–LD4 không thể hiện tương quan tuyến tính rõ, chứng tỏ lượng thông tin mà chúng giữ lại cho bài toán hồi quy là không đáng kể.

V Phân cụm dữ liệu

1 Phương pháp DBScan

a) Kết quả

Quá trình lựa chọn tham số ϵ , $MinPts$ được thực hiện dựa trên phương pháp k-distance graph và chỉ số đánh giá Silhouette Score.

Lựa chọn tham số tối ưu: Dựa trên bảng trên, chúng tôi quyết định lựa chọn bộ tham số: $MinPts = 10$ và $Epsilon (\epsilon) = 2.5$ vì các lý do sau:

Table 2: Tổng hợp kết quả thực nghiệm phân cụm DBSCAN với các tham số khác nhau

MinPts	Epsilon (ϵ)	Số cụm	Điểm nhiễu	Silhouette Score
5	2.0	4	60	0.3525
	2.2	3	37	0.4330
	2.5	1	23	-
	2.8	1	14	-
10	2.2	3	61	0.3841
	2.5	2	25	0.4618
	2.8	1	14	-
15	3.0	1	13	-
	2.2	1	99	-
	2.5	2	39	0.4545
	2.8	1	17	-
20	3.0	1	13	-
	2.2	1	110	-
	2.5	1	68	-
	2.8	1	21	-
	3.0	1	14	-

* Ghi chú: Dấu "-" biểu thị trường hợp chỉ tìm thấy 1 cụm, không tính được Silhouette Score.

- **Chỉ số Silhouette cao nhất:** Trong các trường hợp thuật toán tìm được nhiều hơn 1 cụm, bộ tham số này cho chỉ số Silhouette Score cao nhất (0.4618), chứng tỏ các cụm có độ tách biệt tốt nhất.
- **Số lượng nhiễu hợp lý:** Số điểm nhiễu là 25 (chiếm khoảng 1.8% dữ liệu), đủ nhỏ để không làm mất mát thông tin quan trọng nhưng đủ để loại bỏ các giá trị ngoại lai (outliers).
- **Số lượng cụm:** Kết quả phân ra 2 cụm chính là khá phù hợp để phân tách các nhóm thiết bị (ví dụ: phân khúc phổ thông và cao cấp).

b) Đánh giá

Để kiểm chứng chất lượng của các cụm tìm được, chúng tôi tiến hành đối chiếu nhãn cụm với biến đầu ra là giá sản phẩm (Price_log). Kết quả thống kê chi tiết được trình bày trong Bảng 3.

Table 3: Thống kê mô tả biến giá (Price_log) theo từng cụm DBSCAN

Nhóm (Cluster)	Số lượng	Mean	Std Dev	Min	Max
Cụm 0	1287	4.4352	0.6933	2.4794	6.7165
Cụm 1	37	5.4677	0.5350	4.5658	6.7468
Nhiều (-1)	25	5.5364	0.7893	3.7819	6.6477

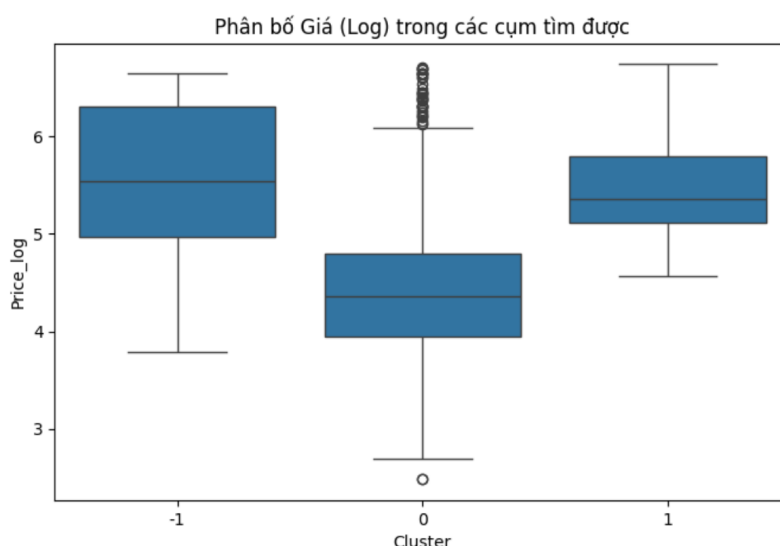


Figure 22: Biểu đồ hộp (Boxplot) so sánh phân bố giá giữa các cụm.

Nhận xét từ dữ liệu thực nghiệm:

- **Sự phân tách về giá:** Cụm 0 đại diện cho nhóm thiết bị phổ thông với mức giá trung bình thấp (4.44). Trong khi đó, Cụm 1 tập trung các thiết bị có mức giá cao hơn hẳn (5.47). Điều này cho thấy thuật toán đã tách biệt thành công hai phân khúc sản phẩm dựa trên cấu hình phần cứng.
- **Độ ổn định:** Cụm 1 có độ lệch chuẩn thấp nhất (0.5350), cho thấy đây là một nhóm rất đồng nhất về cả cấu hình lẫn giá bán (có thể là dòng flagship hoặc high-end).
- **Đặc điểm điểm nhiễu:** Nhóm nhiễu (-1) có độ lệch chuẩn cao nhất (0.7893), phản ánh đúng bản chất của các điểm ngoại lai: giá cả biến động mạnh và không tuân theo quy luật cấu hình thông thường.

c) Trực quan hóa

Do dữ liệu gốc có số chiều lớn (46 chiều), chúng tôi sử dụng kỹ thuật phân tích thành phần chính (PCA) để giảm xuống còn 2 chiều nhằm mục đích hiển thị trên mặt phẳng. Kết quả phân cụm được thể hiện trong Hình 23.

Phân tích biểu đồ:

- Trên không gian PCA, hai cụm dữ liệu tách biệt nhau khá rõ ràng. Cụm 0 (màu chủ đạo) chiếm phần lớn không gian, trong khi Cụm 1 (nhóm nhỏ) tập trung thành một quần thể riêng biệt.
- Các điểm nhiễu (Noise) nằm rải rác ở vùng biên hoặc xen kẽ giữa khoảng tiếp giáp của hai cụm, phù hợp với tính chất của thuật toán DBSCAN dựa trên mật độ.

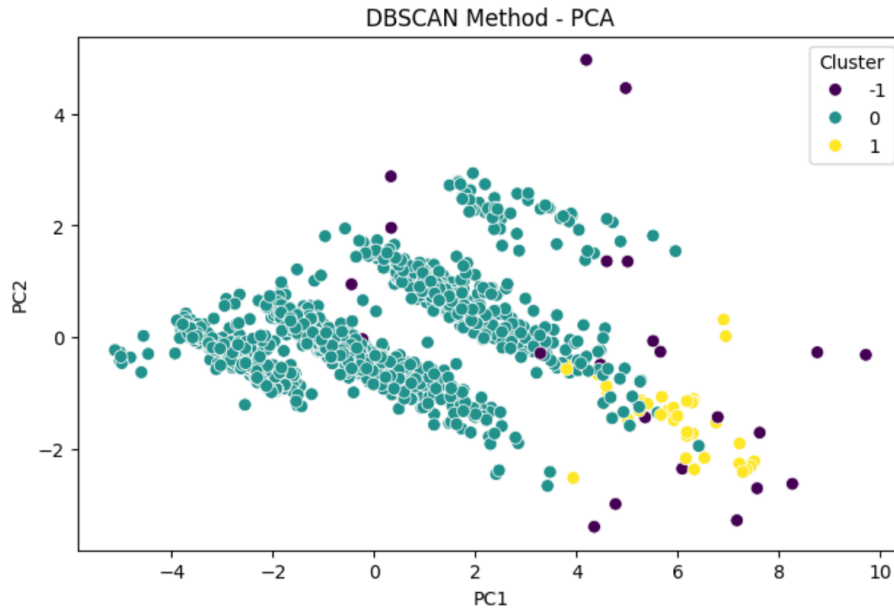


Figure 23: Biểu diễn các cụm dữ liệu trên không gian 2 chiều (PCA).

2 Phương pháp GMM

a) Lựa chọn số cụm tối ưu (k)

Nhóm sử dụng tiêu chuẩn thông tin Bayesian (BIC) để xác định số lượng cụm phù hợp nhất, nhằm cân bằng giữa độ khớp của mô hình và độ phức tạp tính toán.

- Kết quả thực nghiệm cho thấy giá trị BIC đạt cực tiểu (hoặc bão hòa) tại $k = 7$.
- Đây là số lượng cụm tối ưu được lựa chọn cho quá trình huấn luyện.

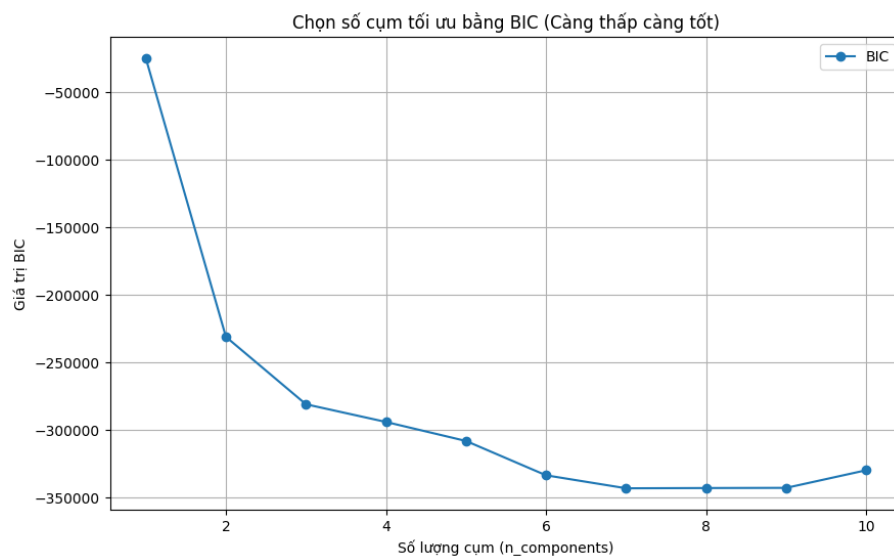


Figure 24: Biểu đồ chỉ số BIC theo số lượng cụm

b) Kết quả và Đánh giá

--- Thống kê biến đầu ra (Price_log) theo từng cụm ---					
Cluster	mean	std	min	max	count
0	3.971178	0.379612	3.135016	5.225736	260
1	3.958697	0.655211	2.690497	5.398113	53
2	4.828001	0.693574	2.479391	6.716462	132
3	4.208554	0.487760	3.135016	5.300904	79
4	5.060763	0.839421	3.135016	6.705399	104
5	5.045887	0.587444	4.025155	6.746752	294
6	4.278385	0.583385	2.479391	6.617269	427

Figure 25: Thống kê biến đầu ra theo từng cụm

Mối quan hệ với biến đầu ra (Giá tiền): Mặc dù quá trình huấn luyện không sử dụng thông tin về giá, kết quả thống kê hậu nghiệm cho thấy sự phân hóa rõ rệt:

- Giá trung bình tăng dần theo các cụm (Cụm 0 thấp nhất → Cụm 2 cao nhất).
- Độ lệch chuẩn về giá trong mỗi cụm nhỏ, cho thấy các thiết bị trong cùng một cụm có mức giá khá tương đồng.

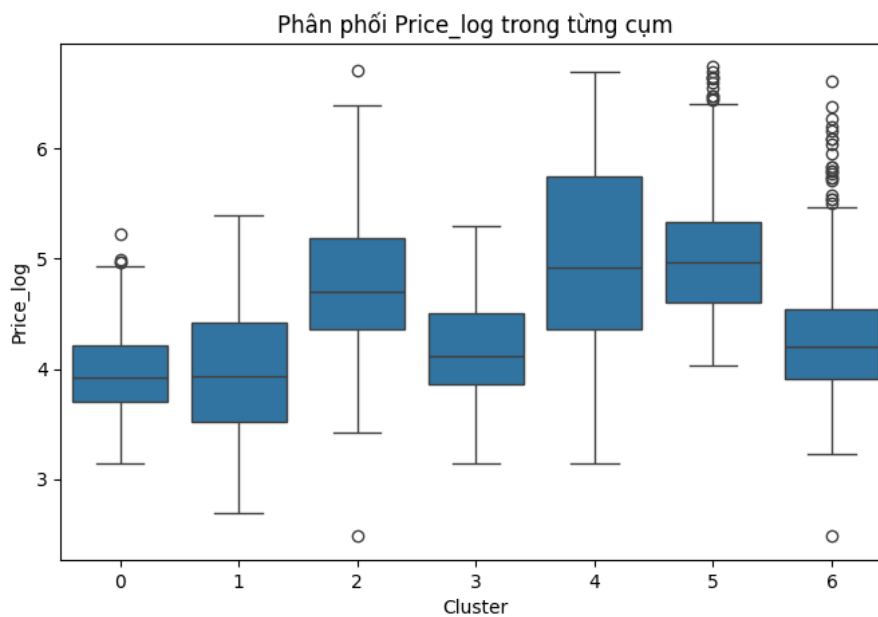


Figure 26: Phân phối giá (Price_log) trong từng cụm GMM

c) Trực quan hóa dữ liệu

Để trực quan hóa kết quả phân cụm trong không gian 2 chiều, nhóm sử dụng kỹ thuật PCA để giảm chiều dữ liệu sau khi đã gán nhãn. Kết quả tại Hình 27 cho thấy các điểm dữ

liệu thuộc cùng một cụm có xu hướng tập trung gần nhau, phản ánh đúng cấu trúc phân khúc của thị trường điện thoại.

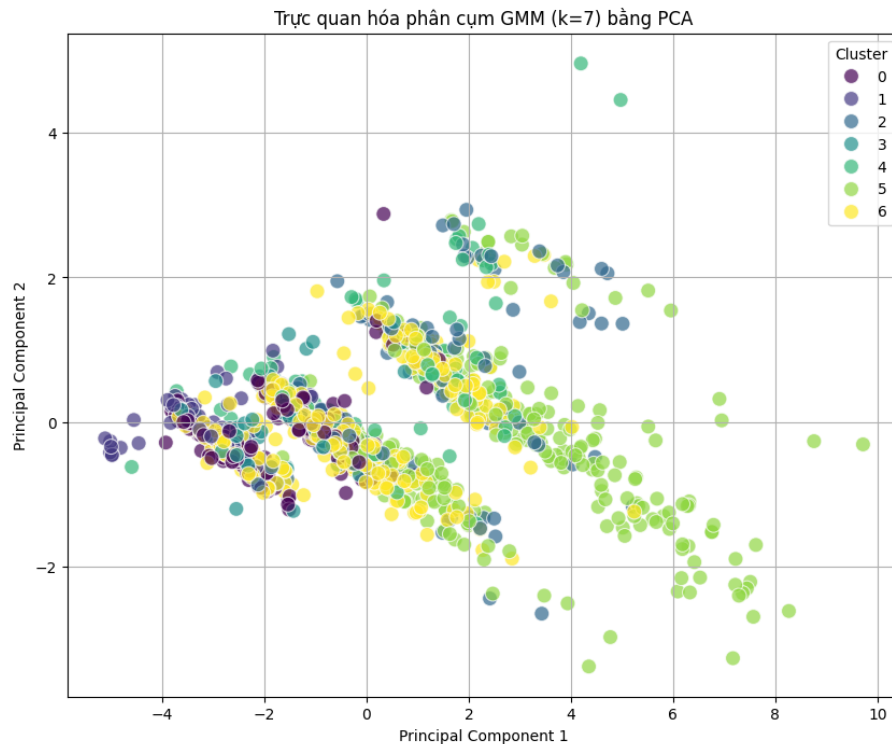


Figure 27: Trực quan hóa các cụm GMM trên không gian PCA 2 chiều

3 Phương pháp Kmeans

a) Thiết lập thực nghiệm

K-Means là thuật toán phân cụm phân hoạch với mục tiêu tối thiểu hóa tổng bình phương khoảng cách giữa các điểm dữ liệu và tâm cụm. Trong thực nghiệm này:

- **Dữ liệu đầu vào:** Dữ liệu đã được chuẩn hóa (StandardScaler) và loại bỏ nhãn giá tiền.
- **Phương pháp đánh giá:** Sử dụng chỉ số **Silhouette Score** để xác định số lượng cụm tối ưu.

b) Lựa chọn số cụm tối ưu (k)

Nhóm đã thử nghiệm số lượng cụm k trong khoảng từ 2 đến 10. Kết quả đánh giá Silhouette Score được thể hiện như sau:

- Biểu đồ cho thấy chỉ số Silhouette đạt giá trị cao nhất tại $k = 2$.

- Do đó, chúng tôi lựa chọn $k = 2$ để thực hiện phân cụm. Điều này gợi ý rằng dữ liệu có xu hướng phân tách tự nhiên thành 2 nhóm lớn (Phân đoạn nhị phân).

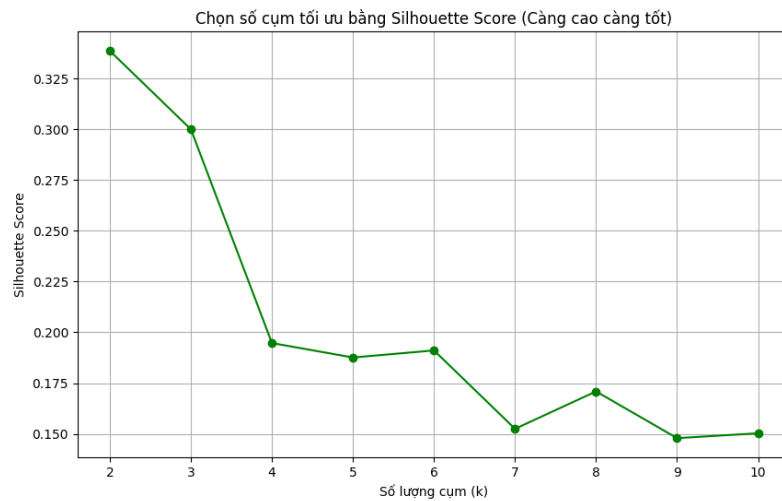


Figure 28: Biểu đồ chỉ số Silhouette theo số lượng cụm

c) Kết quả và Phân tích

Đánh giá định lượng: Mô hình K-Means với $k = 2$ đạt được các chỉ số sau:

- **Silhouette Score:** [0.3385]
- **Inertia (Độ gọn):** [8133.94].

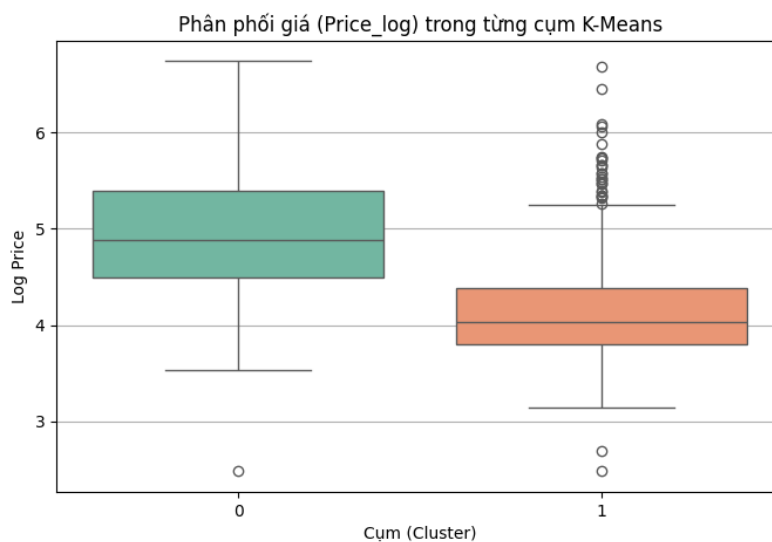


Figure 29: Phân phối giá trong 2 cụm K-Means

d) Trực quan hóa trên không gian giảm chiều

Hình 30 biểu diễn các điểm dữ liệu trên không gian 2 chiều (PCA). Có thể thấy 2 cụm được phân tách khá rõ ràng bởi một ranh giới tuyến tính, phù hợp với bản chất hình học của thuật toán K-Means và kết quả Silhouette Score cao.

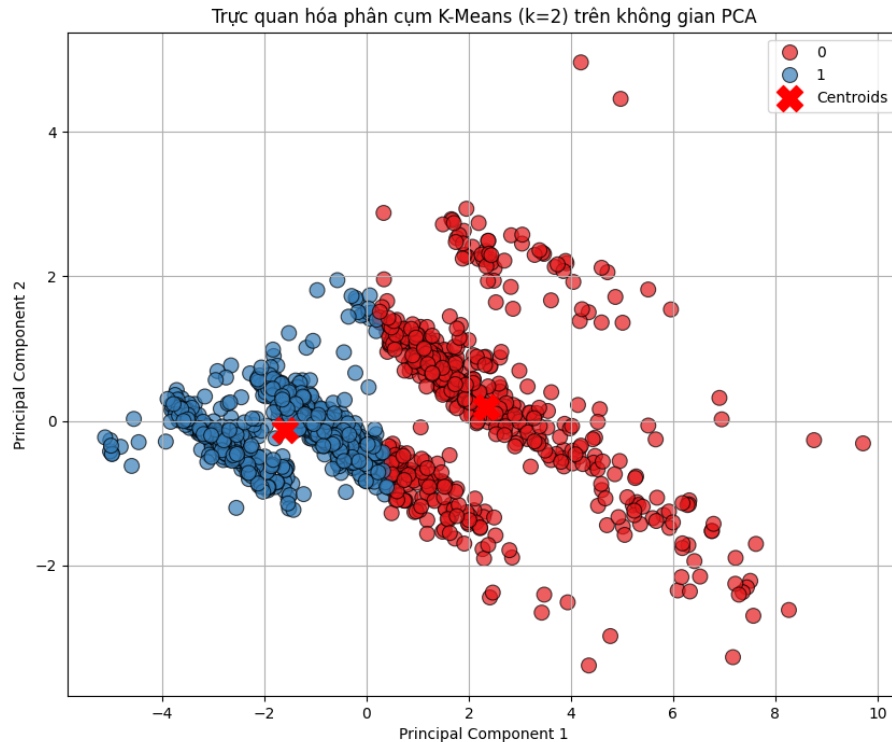


Figure 30: Trực quan hóa phân cụm K-Means ($k = 2$) bằng PCA

VI Kết quả thực nghiệm

1 Các bài toán hồi quy

Tất cả các mô hình được nêu ở phần này được nhóm thực hiện với các kịch bản sau:

- Chia tập huấn luyện và kiểm tra: Các tỉ lệ 4:1, 7:3, 6:4.
- Thực hiện với dữ liệu gốc.
- Thực hiện với dữ liệu đã giảm chiều (4 chiều và 6 chiều).

a) Mô hình K láng giềng gần nhất (K-NN)

Table 4: Kết quả hồi quy KNN với các tỉ lệ chia và dữ liệu khác nhau

(a) Dữ liệu gốc

Tỉ lệ Train:Test	Train			Test		
	MSE	R^2	MAE	MSE	R^2	MAE
4:1	0.1605	0.6884	0.2971	0.2408	0.5783	0.3655
7:3	0.1565	0.7026	0.2933	0.2157	0.5895	0.3459
6:4	0.1872	0.6399	0.3189	0.2196	0.5900	0.3441

(b) Dữ liệu giảm còn 6 chiều (PCA)

Tỉ lệ Train:Test	Train			Test		
	MSE	R^2	MAE	MSE	R^2	MAE
4:1	0.1827	0.6453	0.3151	0.2443	0.5722	0.3732
7:3	0.1953	0.6288	0.3234	0.2131	0.5945	0.3504
6:4	0.1948	0.6254	0.3248	0.2234	0.5830	0.3511

(c) Dữ liệu giảm còn 4 chiều (PCA)

Tỉ lệ Train:Test	Train			Test		
	MSE	R^2	MAE	MSE	R^2	MAE
4:1	0.1808	0.6490	0.3116	0.2364	0.5860	0.3741
7:3	0.2029	0.6145	0.3305	0.2264	0.5692	0.3620
6:4	0.2078	0.6004	0.3356	0.2373	0.5570	0.3614

Đánh giá mô hình KNN

Kết quả huấn luyện và kiểm thử mô hình KNN cho bài toán hồi quy giá điện thoại cho thấy mô hình hoạt động ổn định với độ chính xác trung bình khá. Trên tập dữ liệu gốc, mô hình đạt R^2_{train} từ 0.64 đến 0.70 và R^2_{test} khoảng 0.58–0.59, với chênh lệch khoảng 0.1. Điều này cho thấy mô hình có hiện tượng *overfitting nhẹ*, tức là mô hình học hơi kỹ trên tập huấn luyện và suy giảm hiệu năng khi áp dụng trên tập kiểm thử.

Khi áp dụng kỹ thuật giảm chiều dữ liệu bằng PCA với 6 thành phần chính, mô hình đạt hiệu năng tương đương nhưng ổn định hơn, với R^2_{train} giảm nhẹ còn khoảng 0.63–0.65 và R^2_{test} giữ ổn định ở mức 0.57–0.59. Khoảng cách giữa hai giá trị này giảm xuống chỉ còn 0.04–0.06, chứng tỏ việc giảm chiều đã giúp mô hình tổng quát hóa tốt hơn và giảm

thiếu hiện tượng *overfitting*. Ngược lại, khi tiếp tục giảm xuống 4 thành phần chính, mô hình mất mát thêm thông tin, khiến R^2_{test} giảm xuống khoảng 0.55–0.57, biểu hiện của *underfitting* nhẹ.

Nhìn chung, mô hình KNN hoạt động tốt trên cả dữ liệu gốc và dữ liệu sau PCA, trong đó PCA với 6 thành phần cho hiệu quả cân bằng nhất giữa độ chính xác và khả năng tổng quát hóa. Để giảm hiện tượng overfit, các biện pháp hiệu chỉnh phù hợp bao gồm tăng số lượng lân cận k , sử dụng trọng số khoảng cách ($weights='distance'$) nhằm giảm ảnh hưởng của các điểm xa, và kết hợp đánh giá chéo (*cross-validation*) để tối ưu tham số mô hình.

b) Mô hình Hồi quy tuyến tính

Trước hết, nhóm tiến hành so sánh hai biến thể của mô hình hồi quy tuyến tính: **(1) mô hình có hệ số chệch (bias)** và **(2) mô hình không có hệ số chệch**, nhằm xác định xem thành phần bias có ảnh hưởng đáng kể đến độ chính xác dự đoán hay không. Thực nghiệm được thực hiện trên **tập dữ liệu gốc** với tỉ lệ chia **train:test = 4:1**.

Mô hình 1: Có hệ số chệch (bias)

Hiệu suất trên tập **train**:

$$MSE = 0.1881, \quad R^2 = 0.6349, \quad MAE = 0.3238$$

Hiệu suất trên tập **test**:

$$MSE = 0.2209, \quad R^2 = 0.6131, \quad MAE = 0.3571$$

Mô hình 2: Không có hệ số chệch (bias)

Hiệu suất trên tập **train**:

$$MSE = 1.0086, \quad R^2 = -0.9581, \quad MAE = 0.6601$$

Hiệu suất trên tập **test**:

$$MSE = 0.9024, \quad R^2 = -0.5804, \quad MAE = 0.6547$$

Nhận xét: Kết quả cho thấy mô hình **có hệ số chệch bias** cho hiệu suất vượt trội rõ rệt so với mô hình không bias. Cụ thể, giá trị R^2 của mô hình có bias đạt khoảng 0.63 trên

cả tập train và test, trong khi mô hình không bias cho R^2 âm, cho thấy mô hình không có khả năng mô tả được quan hệ tuyến tính trong dữ liệu. Điều này chứng minh rằng việc thêm hệ số chệch giúp mô hình linh hoạt hơn trong việc điều chỉnh mặt phẳng hồi quy, tránh việc buộc đường hồi quy phải đi qua gốc toạ độ — vốn không phù hợp với phân bố thực tế của dữ liệu.

Từ kết quả này, nhóm loại bỏ phương án “không bias” và sử dụng **mô hình có hệ số chệch** cho các thực nghiệm tiếp theo (trên các tỉ lệ chia khác và dữ liệu giảm chiều PCA).

Table 5: Kết quả hồi quy tuyến tính với các tỉ lệ chia và dữ liệu khác nhau

(a) Dữ liệu gốc						
Tỉ lệ Train:Test	Train			Test		
	MSE	R^2	MAE	MSE	R^2	MAE
4:1	0.1881	0.6349	0.3238	0.2209	0.6131	0.3571
7:3	0.1943	0.6308	0.3279	0.1964	0.6262	0.3391
6:4	0.1899	0.6348	0.3234	0.2035	0.6200	0.3394

(b) Dữ liệu giảm còn 4 chiều (PCA)						
Tỉ lệ Train:Test	Train			Test		
	MSE	R^2	MAE	MSE	R^2	MAE
4:1	0.2176	0.5775	0.3463	0.2457	0.5698	0.3810
7:3	0.2240	0.5743	0.3515	0.2206	0.5802	0.3596
6:4	0.2174	0.5819	0.3466	0.2304	0.5699	0.3592

(c) Dữ liệu giảm còn 6 chiều (PCA)						
Tỉ lệ Train:Test	Train			Test		
	MSE	R^2	MAE	MSE	R^2	MAE
4:1	0.2115	0.5895	0.3418	0.2419	0.5764	0.3755
7:3	0.2188	0.5843	0.3470	0.2143	0.5921	0.3542
6:4	0.2117	0.5929	0.3420	0.2263	0.5775	0.3556

Nhận xét và đánh giá kết quả

Dựa trên bảng kết quả ở trên, có thể rút ra một số nhận xét như sau:

- Khi so sánh giữa các tỉ lệ chia, ta nhận thấy tỉ lệ **7:3** cho kết quả tốt nhất về cả ba chỉ số (MSE, R^2 , MAE). Cụ thể, với dữ liệu gốc, mô hình đạt $R^2_{\text{test}} = 0.6262$, cao

hơn so với 0.6131 ở tỉ lệ 4:1 và 0.62 ở tỉ lệ 6:4. Điều này cho thấy việc giữ lại nhiều dữ liệu huấn luyện hơn (70%) giúp mô hình học được quan hệ tuyến tính tốt hơn mà không bị overfit đáng kể.

- Khi áp dụng **PCA để giảm chiều dữ liệu**, hiệu suất của mô hình giảm nhẹ. Cụ thể, ở PCA-4D và PCA-6D, giá trị R^2 dao động quanh mức 0.57–0.59, thấp hơn khoảng 0.04–0.05 so với dữ liệu gốc. Nguyên nhân là việc giảm chiều làm mất đi một phần thông tin đặc trưng quan trọng, khiến mô hình không thể khớp dữ liệu tốt như ban đầu. Tuy nhiên, sự chênh lệch không quá lớn, chứng tỏ PCA vẫn giữ lại phần lớn thông tin cần thiết.
- So sánh giữa PCA-4D và PCA-6D, mô hình **PCA-6D** cho hiệu suất tốt hơn (ví dụ $R^2_{\text{test}} = 0.5921$ ở tỉ lệ 7:3 so với 0.5802 của PCA-4D), cho thấy việc giữ lại 6 thành phần chính giúp cân bằng giữa giảm nhiễu và bảo toàn thông tin.
- Xét toàn bộ kết quả, ta thấy sai lệch giữa tập train và test không lớn (chênh lệch R^2 chỉ khoảng 0.02–0.03), chứng tỏ mô hình **không xảy ra overfitting đáng kể**. Tuy nhiên, vẫn có thể cải thiện bằng các biện pháp hiệu chỉnh như *Ridge Regression* hoặc *Lasso Regression* để giảm phương sai mô hình khi kích thước dữ liệu tăng.

Tóm lại, kịch bản **Linear Regression có bias, dữ liệu gốc, tỉ lệ train:test = 7:3** là trường hợp cho kết quả tốt nhất với $R^2_{\text{test}} = 0.6262$, $MSE = 0.1964$, $MAE = 0.3391$. Do đó, mô hình này được chọn để phân tích sâu hơn về các hệ số hồi quy và phần dư. Phương pháp PCA tuy làm giảm nhẹ độ chính xác nhưng lại giúp mô hình đơn giản hơn và ít chịu ảnh hưởng bởi nhiễu, thích hợp trong các ứng dụng cần tối ưu hoá tốc độ xử lý.

Tiếp theo ta sẽ đánh giá tương quan phần dư, cũng như hệ số của mô hình được cho là tốt nhất.

Table 6: Các hệ số (β) của mô hình

Hệ số	Giá trị	Hệ số	Giá trị
β_0 (bias)	4.6347	β (RAM)	0.2079
β (OSs_iOS)	1.256	β (OSs_Other)	0.2891
β (Resolution y)	0.1492	β (Resolution x)	0.1454
β (Storage)	0.0789	β (Rear camera)	0.0534
β (4G_Yes)	0.0161	β (Battery capacity)	0.0134
β (Screen size)	0.01262	β (Front camera)	-0.0666
β (CPU_8_10)	-0.0122	β (CPU_4_6)	-0.0205
β (SIM_2)	-0.1873		

Các biến có hệ số dương (ảnh hưởng làm tăng giá):

- **Screen size (0.0126):** Màn hình càng lớn, giá có xu hướng tăng nhẹ.
- **Resolution x (0.1454) và Resolution y (0.1492):** Độ phân giải cao hơn giúp cải thiện hiển thị → tăng giá.
- **Storage (0.0789):** Dung lượng lưu trữ lớn hơn dẫn đến giá cao hơn.
- **Rear camera (0.0534):** Camera sau có độ phân giải cao hơn thường làm giá tăng.
- **RAM (0.2079):** RAM là yếu tố ảnh hưởng mạnh nhất trong nhóm thông số kỹ thuật, mỗi đơn vị RAM tăng kéo theo giá tăng đáng kể.
- **Battery capacity (0.0134):** Pin lớn hơn giúp điện thoại hấp dẫn hơn, nhưng ảnh hưởng ở mức nhẹ.
- **4G_Yes (0.0161):** Có hỗ trợ 4G làm giá tăng nhẹ, thể hiện tính năng cơ bản của smartphone hiện đại.
- **OSs_Other (0.2891):** Một số hệ điều hành khác (ví dụ các bản Android đặc biệt) có thể tăng giá trị thiết bị.
- **OSs_iOS (1.256):** Là hệ số dương lớn nhất — cho thấy iPhone thường có giá cao vượt trội so với các máy khác cùng cấu hình.

Các biến có hệ số âm (ảnh hưởng làm giảm giá):

- **Front camera (-0.0666):** Có thể do thị trường đánh giá camera trước ít quan trọng hơn, hoặc tương quan nghịch với camera sau.
- **SIM_2 (-0.1873):** Máy có 2 SIM thường thuộc nhóm tầm trung, làm giá trung bình thấp hơn.
- **CPU_4_6 (-0.0205) và CPU_8_10 (-0.0122):** Các nhóm CPU này có thể mang tính phân loại, trong đó giá trị âm cho thấy nhóm đó rẻ hơn so với nhóm tham chiếu (có thể là CPU 2 nhân hoặc cao cấp hơn).

Nhận xét tổng quát:

- Mô hình cho thấy các yếu tố kỹ thuật như RAM, độ phân giải, và hệ điều hành iOS là những yếu tố tác động mạnh nhất đến giá điện thoại.
- Một số đặc điểm phổ thông (2 SIM, camera trước lớn) lại có ảnh hưởng âm, phản ánh đúng xu hướng thị trường.

- Hệ số bias ($\beta_0 = 4.635$) biểu diễn giá cơ bản (theo thang log hoặc chuẩn hóa) khi các biến đầu vào bằng 0.

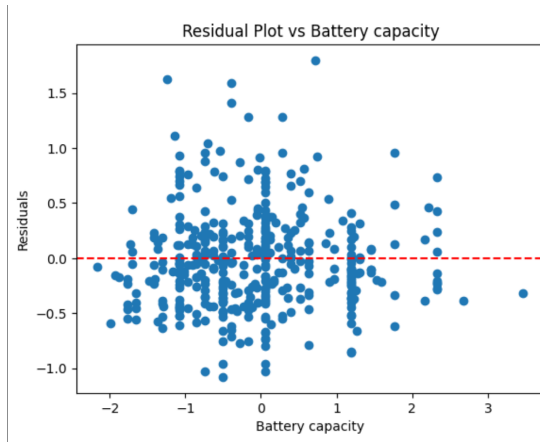


Figure 31: Biểu đồ Phần dư so với 'Battery capacity'



Figure 32: Biểu đồ tương quan Phần dư so với 'Screen size'

Cả 2 biểu đồ đều cho thấy mô hình hồi quy là **phù hợp** và giả định của mô hình được đáp ứng.

- **Tính ngẫu nhiên (Tốt):** Trong cả hai biểu đồ, các điểm dữ liệu (phần dư) đều phân tán một cách ngẫu nhiên xung quanh đường $y = 0$ (đường gạch đỏ).
- **Không có tính phi tuyến (Tốt):** Không có bất kỳ mẫu hình đường cong (như parabol hay hình chữ U) nào, cho thấy giả định về mối quan hệ tuyến tính là đúng.
- **Phương sai đồng đều (Tốt):** Không có dấu hiệu rõ rệt của "hình cái phễu" (funnel shape). Độ "loè" của các đám mây điểm là tương đối nhất quán trên toàn bộ dải giá trị của các biến đầu vào.

Kết luận: Cả hai biểu đồ đều xác nhận tính hợp lệ của mô hình. Biểu đồ của 'Battery capacity' (Hình 18) là một ví dụ rất điển hình về phần dư ngẫu nhiên. Biểu đồ của 'Screen size' (Hình 17) cũng cho thấy sự phù hợp; sự tập trung của các điểm theo chiều dọc (đặc biệt quanh giá trị 0 và 1) có thể là do sự phân bố của chính dữ liệu 'Screen size' chứ không phải là lỗi của mô hình.

c) Nhận xét, so sánh giữa 2 mô hình

Từ hai bảng kết quả trên, ta có thể rút ra các nhận xét sau:

- **Hiệu suất tổng thể:** Mô hình **Hồi quy Tuyến tính (Linear Regression)** cho **hiệu suất tổng thể tốt hơn** so với Hồi quy KNN. Ở kịch bản tốt nhất (dữ liệu gốc, chia

7:3), Hồi quy Tuyến tính đạt $R^2 \approx 0.626$ trên tập test, trong khi Hồi quy KNN chỉ đạt $R^2 \approx 0.590$.

- **Vấn đề Overfitting (Quá khớp):**

- **Hồi quy KNN (Bảng 2)** thể hiện dấu hiệu **overfitting** rõ rệt, đặc biệt là trên dữ liệu gốc (phần a). Hiệu suất trên tập Train (ví dụ: $R^2 = 0.7026$) cao hơn đáng kể so với hiệu suất trên tập Test ($R^2 = 0.5895$).
- **Hồi quy Tuyến tính (Bảng 3)** hoạt động **cực kỳ ổn định** và **không có dấu hiệu overfitting**. Hiệu suất trên tập Train và Test ở tất cả các kịch bản đều rất gần nhau (ví dụ: R^2 Train=0.6308 và Test=0.6262).

- **Ảnh hưởng của Giảm chiều (PCA):**

- **Đối với KNN:** Việc áp dụng PCA (cả 4 và 6 chiều) đã giúp **giảm bớt hiện tượng overfitting** (thu hẹp khoảng cách giữa R^2 Train và R^2 Test), nhưng không cải thiện được hiệu suất tổng thể trên tập Test (vẫn dao động quanh $R^2 \approx 0.57 - 0.59$).
 - **Đối với Hồi quy Tuyến tính:** Việc áp dụng PCA (cả 4 và 6 chiều) đã làm **giảm hiệu suất** của mô hình. R^2 trên tập Test giảm từ ≈ 0.62 xuống còn $\approx 0.57 - 0.59$. Điều này cho thấy Hồi quy Tuyến tính đã tận dụng được thông tin từ tất cả các đặc trưng gốc, và việc giảm chiều đã làm mất đi thông tin hữu ích.
- **Độ ổn định của tỷ lệ chia:** Mô hình Hồi quy Tuyến tính cho kết quả rất nhất quán, bất kể tỷ lệ chia là 4:1, 7:3 hay 6:4. Ngược lại, mô hình KNN có vẻ nhạy cảm hơn với cách chia (ví dụ, R^2 Train giảm mạnh ở tỷ lệ 6:4).

Kết luận chung

Mô hình **Hồi quy Tuyến tính** (Linear Regression) sử dụng bộ **dữ liệu gốc** (không giảm chiều) là phương pháp tốt nhất, ổn định nhất và không bị overfitting cho bài toán này. Mô hình KNN phức tạp hơn, bị overfitting và không mang lại hiệu quả bằng.

2 Các bài toán phân loại

Tất cả các mô hình được nêu ở phần này được nhóm thực hiện với các kịch bản sau:

- Đầu ra được phân làm 3 hoặc 4 khoảng có cỡ mẫu xấp xỉ nhau dựa theo mức phân vị.
- Chia tập huấn luyện và kiểm tra: Các tỉ lệ 4:1, 7:3, 6:4.

- Thực hiện với dữ liệu gốc.
- Thực hiện với dữ liệu đã giảm chiều (1/3 số chiều so với số chiều gốc, tức khoảng 15 chiều).

a) **Mô hình Naive Bayes**

Table 7: Kết quả thực nghiệm mô hình Naive Bayes trên các tập dữ liệu

Trường hợp	Loại dữ liệu	Tỷ lệ (Train:Test)	Accuracy
3 Khoảng	Gốc	6:4	0.6185
		7:3	0.6025
		8:2	0.5889
	PCA	6:4	0.7130
		7:3	0.7012
		8:2	0.6963
4 Khoảng	Gốc	6:4	0.5222
		7:3	0.5259
		8:2	0.5259
	PCA	6:4	0.6037
		7:3	0.6222
		8:2	0.6704

Dựa trên bảng số liệu thống kê độ chính xác (Accuracy) của mô hình Naive Bayes, chúng tôi rút ra một số nhận xét quan trọng như sau:

- **Tác động của giảm chiều dữ liệu (PCA):** Việc áp dụng kỹ thuật phân tích thành phần chính (PCA) đã cải thiện đáng kể hiệu suất của mô hình. Trong tất cả các trường hợp thử nghiệm, dữ liệu đã qua xử lý PCA luôn cho độ chính xác cao hơn so với dữ liệu gốc.
 - Cụ thể, trong trường hợp *3 Khoảng*, độ chính xác tăng từ mức trung bình ≈ 0.60 lên tới ≈ 0.70 .
 - Điều này cho thấy PCA đã loại bỏ hiệu quả các đặc trưng dư thừa và nhiễu, giúp thỏa mãn tốt hơn giả định về sự độc lập giữa các biến của thuật toán Naive Bayes.
- **Ảnh hưởng của số lượng khoảng phân chia:** Mô hình hoạt động hiệu quả hơn hẳn khi dữ liệu được chia thành **3 khoảng** so với **4 khoảng**.
 - Độ chính xác cao nhất đạt được ở trường hợp 3 khoảng là 0.7130, trong khi ở trường hợp 4 khoảng chỉ đạt 0.6704.
 - Sự sụt giảm hiệu suất khi tăng số lượng khoảng cho thấy độ phức tạp của bài toán tăng lên, khiến mô hình khó phân tách ranh giới giữa các lớp dữ liệu hơn.

- **Tác động của tỷ lệ phân chia dữ liệu (Train:Test):**

- Đối với trường hợp 3 Khoảng (PCA), tỷ lệ chia **6:4** mang lại kết quả tốt nhất (0.7130).
- Ngược lại, ở trường hợp 4 Khoảng (PCA), tỷ lệ **8:2** lại chiếm ưu thế (0.6704). Điều này ngụ ý rằng khi bài toán phân lớp trở nên phức tạp hơn (từ 3 lên 4 nhãn), mô hình cần lượng dữ liệu huấn luyện lớn hơn để trích xuất được các đặc trưng quan trọng.

- **Kết luận:** Cấu hình tối ưu nhất cho bài toán này khi sử dụng Naive Bayes là: Phân chia dữ liệu thành **3 khoảng**, sử dụng phương pháp **PCA**, và tỷ lệ phân chia tập dữ liệu là **6:4**.

b) Mô hình hồi quy SoftMax

Table 8: Kết quả thực nghiệm mô hình SoftMax Regression trên các tập dữ liệu

Trường hợp	Loại dữ liệu	Tỷ lệ (Train:Test)	Accuracy
3 Khoảng	Gốc	6:4	0.6704
		7:3	0.6716
		8:2	0.6593
	PCA	6:4	0.6556
		7:3	0.6691
		8:2	0.6407
4 Khoảng	Gốc	6:4	0.5667
		7:3	0.5926
		8:2	0.5889
	PCA	6:4	0.5407
		7:3	0.5778
		8:2	0.5815

Dựa trên Bảng 8, nhóm đưa ra các phân tích chi tiết về hiệu năng của mô hình Softmax Regression như sau:

So sánh giữa dữ liệu Gốc (Original) và dữ liệu giảm chiều (PCA)

- **Mức độ sụt giảm độ chính xác:** Mức độ sụt giảm là không đáng kể giữa trường hợp dữ liệu gốc và đã giảm chiều, dao động trong khoảng từ 0.2% đến 2.6%.
- **Đánh giá hiệu quả:** Việc giảm số chiều dữ liệu xuống còn 1/3 (từ 46 xuống 15) giúp giảm đáng kể chi phí tính toán và thời gian huấn luyện mô hình, trong khi vẫn duy trì được độ chính xác ở mức chấp nhận được. Điều này chứng tỏ PCA đã trích xuất thành công các đặc trưng quan trọng nhất của dữ liệu.

Ảnh hưởng của tỷ lệ phân chia Train:Test

- **Tỷ lệ tối ưu:** Tỷ lệ phân chia **7:3** cho kết quả ổn định và tốt nhất trong đa số các trường hợp (đạt 67.16% với bài toán 3 nhãn và 59.26% với bài toán 4 nhãn trên dữ liệu gốc).
- **Phân tích tỷ lệ 8:2:** Mặc dù tỷ lệ train 80% cung cấp nhiều dữ liệu học hơn, nhưng độ chính xác lại có xu hướng giảm nhẹ so với tỷ lệ 7:3. Nguyên nhân có thể do tập kiểm thử (Test set) trở nên quá nhỏ (chỉ 20%), dẫn đến kết quả đánh giá kém ổn định và phụ thuộc nhiều vào phân phối ngẫu nhiên của các mẫu trong tập test.

Ảnh hưởng của số lượng nhãn phân loại (3 khoảng vs 4 khoảng)

- **Sự suy giảm hiệu năng:** Có sự sụt giảm mạnh về độ chính xác khi tăng số lượng lớp từ 3 lên 4 (giảm khoảng 8% – 10%). Cụ thể, độ chính xác tốt nhất giảm từ $\approx 67\%$ xuống còn $\approx 59\%$.
- **Nguyên nhân:** Việc chia nhỏ biến mục tiêu (Giá) thành nhiều khoảng hơn làm cho biên giới giữa các lớp trở nên hẹp và khó phân biệt hơn. Các mẫu dữ liệu nằm ở vùng ranh giới giữa các khoảng giá (ví dụ: ranh giới giữa "Rẻ" và "Trung bình rẻ") có đặc trưng kỹ thuật rất giống nhau, khiến mô hình Softmax – vốn là một bộ phân loại tuyến tính – gặp khó khăn trong việc tách biệt.

Kết luận chung: Mô hình Softmax Regression hoạt động hiệu quả nhất trên bài toán phân loại 3 mức giá với tỷ lệ chia dữ liệu 7:3. Phương pháp PCA được khuyến nghị sử dụng khi cần tối ưu hóa tốc độ xử lý trên các bộ dữ liệu lớn, do sự đánh đổi về độ chính xác là không lớn. Tuy nhiên, để giải quyết bài toán phân loại chi tiết hơn (4 mức giá trở lên), cần cân nhắc sử dụng các mô hình phi tuyến tính mạnh hơn (như SVM hoặc Random Forest) để cải thiện độ chính xác.

VII Tổng kết và Hướng phát triển

1 Tóm tắt kết quả

Nhóm đã thực hiện phân cụm bộ dữ liệu với ba phương pháp Kmeans, GMM, DBScan.

Đối với bài toán hồi quy nhóm đã xây dựng và đánh giá hai mô hình học máy – **Linear Regression** và **K-Nearest Neighbors (KNN)** – cho bài toán dự đoán **giá điện thoại**. Các mô hình được đánh giá trên dữ liệu gốc và dữ liệu giảm chiều bằng PCA với 4 và 6 thành phần, trên các tỷ lệ chia Train:Test (4:1, 7:3, 6:4).

Đối với bài toán phân loại, đầu ra được gom nhóm thành các khoảng để chuyển về bài toán phân loại. Nhóm đã thực hiện với hai mô hình học máy **Naive Bayes** và **hồi quy**

SoftMax. Các mô hình đều thử nghiệm các các kịch bản gom nhóm khác nhau trên bộ dữ liệu gốc, giảm chiều và các tỉ lệ chia train test khác nhau.

Đối với bài toán hồi quy, kết quả thực nghiệm cho thấy mô hình **Linear Regression** có **hệ số chệch (bias)** trên dữ liệu gốc với tỉ lệ Train:Test = 7:3 đạt hiệu suất tốt nhất và ổn định.

Ở bài toán phân loại, kết quả thực nghiệm tốt nhất đạt độ chính xác 0.713, mô hình Naive Bayes với kịch bản chia thành 3 khoảng đầu ra, bộ dữ liệu PCA, tỉ lệ train:test 6:4.

2 Phân tích và Đánh giá

a) Hiệu suất Linear Regression

- **Độ ổn định và không overfitting:** Mô hình có $R^2_{\text{train}} \approx 0.631$ và $R^2_{\text{test}} \approx 0.626$, cho thấy khả năng khái quát hóa tốt, không bị quá khớp.
- **Tác động của PCA:** Áp dụng PCA (4D hoặc 6D) làm giảm hiệu suất nhẹ ($R^2_{\text{test}} \approx 0.58$), chứng tỏ mô hình tuyến tính tận dụng tốt thông tin từ bộ đặc trưng gốc.
- **Phân tích phần dư:** Biểu đồ residual cho thấy phần dư phân bố ngẫu nhiên quanh 0, xác nhận các giả định tuyến tính được đáp ứng.

b) Hiệu suất KNN

- **Hiện tượng overfitting:** KNN có $R^2_{\text{train}} \approx 0.70$ nhưng $R^2_{\text{test}} \approx 0.59$ (tỷ lệ 7:3), cho thấy hiện tượng quá khớp rõ rệt.
- **Tác động của PCA:** PCA giúp giảm nhẹ overfitting, nhưng không cải thiện hiệu suất tổng thể trên tập Test.

c) Hiệu suất Naive Bayes

- Mô hình có độ chính xác từ 50 đến 70 phần trăm ở các kịch bản.
- Tác động của PCA: Kết quả cho thấy độ chính xác tăng 0.1. Có thể PCA đã giúp làm giảm nhiễu.

d) Hiệu suất SoftMax

- Mô hình có độ chính xác từ 54 đến 67 phần trăm ở các kịch bản.
- Tác động của PCA: Kết quả cho thấy độ chính xác giảm nhẹ. Thông tin bị mất đi một chút.

3 Kết luận

Ở bài toán hồi quy mô hình **Linear Regression có bias trên dữ liệu gốc** là phương pháp tối ưu cho bài toán dự đoán giá điện thoại, vừa cho hiệu suất cao ($R_{\text{test}}^2 \approx 0.626$) vừa ổn định. Các đặc trưng ảnh hưởng mạnh nhất đến giá điện thoại là **OSs_iOS**, **RAM**, và **Resolution**.

Ở bài toán phân loại Naive Bayes là phương án tốt nhất cho bài toán này.

4 Hướng phát triển

Mặc dù mô hình Linear Regression hoạt động tốt, vẫn còn khoảng 38% biến động của dữ liệu chưa được giải thích, gợi ý rằng mối quan hệ giữa đặc trưng và giá có thể phi tuyến. Các hướng phát triển tương lai bao gồm:

- Thử nghiệm các mô hình phi tuyến như **Random Forest**, **Gradient Boosting** (XGBoost) hoặc mạng nơ-ron (Neural Networks).
- Nâng cao kỹ thuật **Feature Engineering**, tạo thêm các đặc trưng tương tác, thay vì chỉ giảm chiều.

References

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [2] I. T. Jolliffe and J. Cadima, *Principal Component Analysis: A Review and Recent Developments*, Philosophical Transactions of the Royal Society A, 374(2065), 2016.
- [3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Springer, 2009.
- [4] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 12, pp. 2825–2830, 2011.
- [5] Pratik Garai, *Mobile Phone Specifications and Prices Dataset*, Kaggle, 2020. <https://www.kaggle.com/datasets/pratikgarai/mobile-phone-specifications-and-prices>
- [6] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.